



# Learning Relational Representations from Relational Databases

*A Graph Representation Perspective on Relational Deep Learning*

2026-08-26  
Eunjae Son  
**MINDS Lab**

## ■ Introduction

- ☐ What Is a Relational Database(RDB)?
- ☐ Traditional Approach

## ■ Relational Deep Learning (RDL)

- ☐ How RDL Works
- ☐ Graph Representation in RDL

## ■ Rethinking Graph Representation

- ☐ *What Makes a Desired Graph for Relational Deep Learning?*

## ■ Conclusion

# Introduction

■ Much of the world's important data is stored in relational databases



E-commerce



Social Media



Healthcare



Open-source Scientific  
Repositories

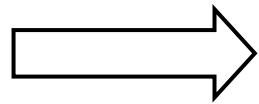
...

# What Is a Relational Database(RDB)?

- Multiple tables linked by Primary–Foreign Key relationships

| Customers                                                                           | Items                                                                               |
|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|    |    |
|    |    |
|    |    |
|    |    |
|   |   |
|  |  |

Normalization



Numbers denote IDs, while icons indicate full entity information

| <u>C_id</u> | Attr.                                                                               |
|-------------|-------------------------------------------------------------------------------------|
| 1           |  |
| 2           |  |
| 3           |  |

Customers

| <u>T_id</u> | C_id | I_id |
|-------------|------|------|
| 1           | 1    | 1    |
| 2           | 3    | 2    |
| 3           | 2    | 1    |
| 4           | 1    | 3    |
| 5           | 3    | 3    |
| 6           | 3    | 1    |

Transactions

| <u>I_id</u> | Attr.                                                                                |
|-------------|--------------------------------------------------------------------------------------|
| 1           |   |
| 2           |   |
| 3           |  |

Items

# Prediction Tasks on Relational Databases

■ RDB prediction supports real-world decision-making



E-commerce



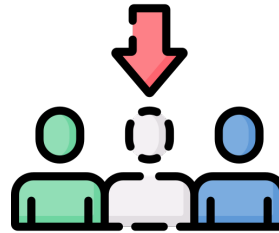
Social Media



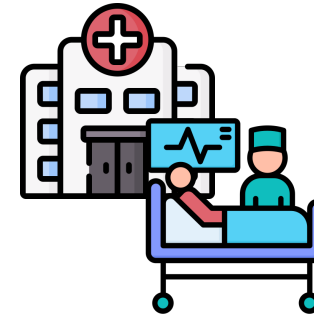
Healthcare



Sales Forecast



Churn Prediction

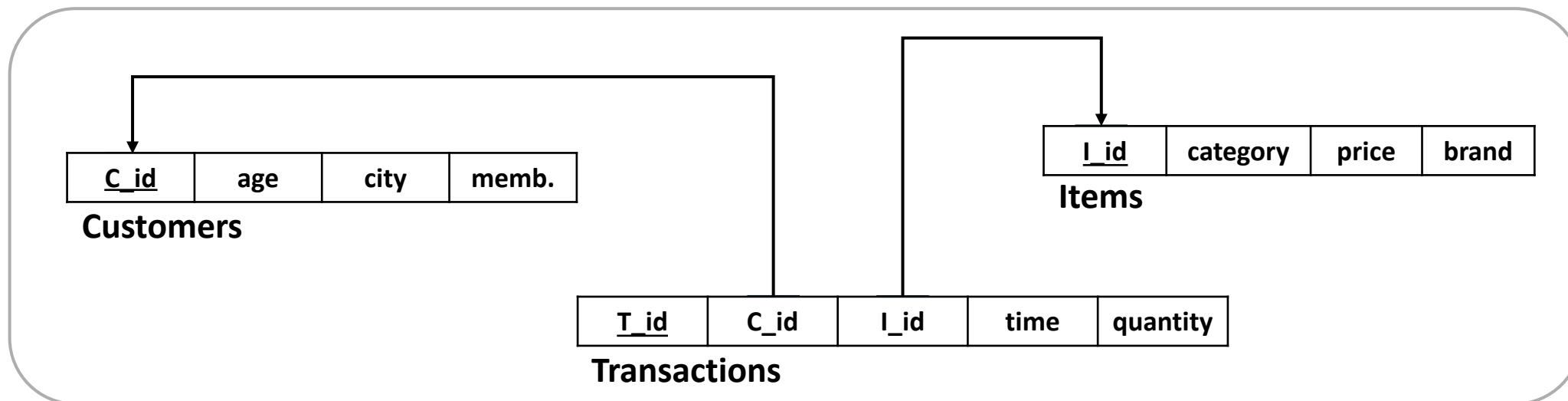


Risk After Discharge

# Traditional Approach

## ■ Transform relational data into a single-table representation

Relational  
Database



Task: Will customer C purchase item I in the next 90 days?



**Feature Engineering:** Join + Aggregation

Single  
Table

|     |       |            |                 |                          |                          |
|-----|-------|------------|-----------------|--------------------------|--------------------------|
| age | memb. | item_price | #past purchases | #same category purchases | days since last purchase |
|-----|-------|------------|-----------------|--------------------------|--------------------------|



**Tabular ML**  
*XGBoost · LightGBM*

# Limitations of the Traditional Approach

## ■ Manual Feature Engineering

Relational  
Database



Task: Will customer purchase item in the next 7 days?



**Feature Engineering:** Join + Aggregation

Single  
Table

|     |       |            |                 |                          |                          |
|-----|-------|------------|-----------------|--------------------------|--------------------------|
| age | memb. | item_price | #past purchases | #same category purchases | days since last purchase |
|-----|-------|------------|-----------------|--------------------------|--------------------------|

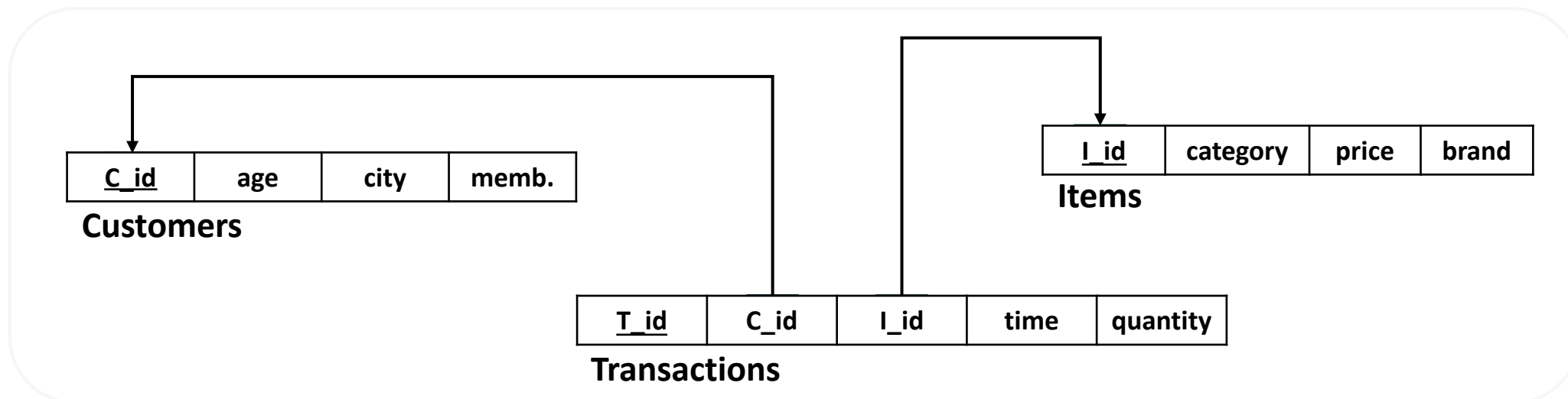


**Tabular ML**  
XGBoost · LightGBM

# Limitations of the Traditional Approach

## ■ Loss of Relational Structure

Relational  
Database



Task: Will customer C purchase item I in the next 7 days?



**Feature Engineering:** Join + Aggregation

Single  
Table

|     |       |            |                 |                          |                          |
|-----|-------|------------|-----------------|--------------------------|--------------------------|
| age | memb. | item_price | #past purchases | #same category purchases | days since last purchase |
|-----|-------|------------|-----------------|--------------------------|--------------------------|

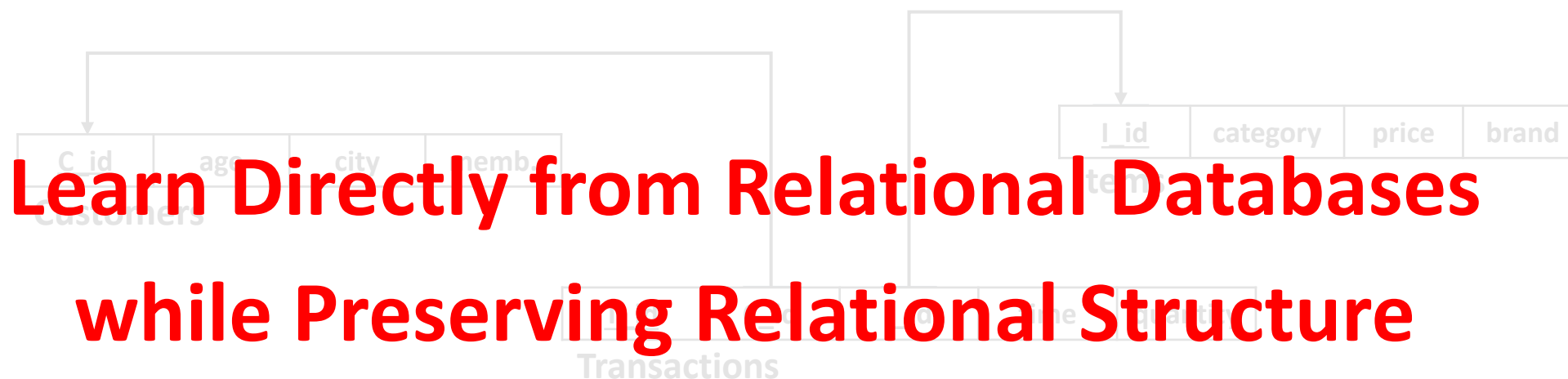


**Tabular ML**  
XGBoost · LightGBM

# Limitations of Previous Approach

## ■ Limitations of Previous Approach: Loss of Relational Structure

Relational  
Database



Task: Will customer C purchase item I in the next 7 days?



**Feature Engineering:** Join + Aggregation

Single  
Table

|     |       |            |                 |                          |                          |
|-----|-------|------------|-----------------|--------------------------|--------------------------|
| age | memb. | item_price | #past purchases | #same category purchases | days since last purchase |
|-----|-------|------------|-----------------|--------------------------|--------------------------|



**Tabular ML**  
XGBoost · LightGBM

# Relational Deep Learning

## - Graph Representation Learning on Relational Databases

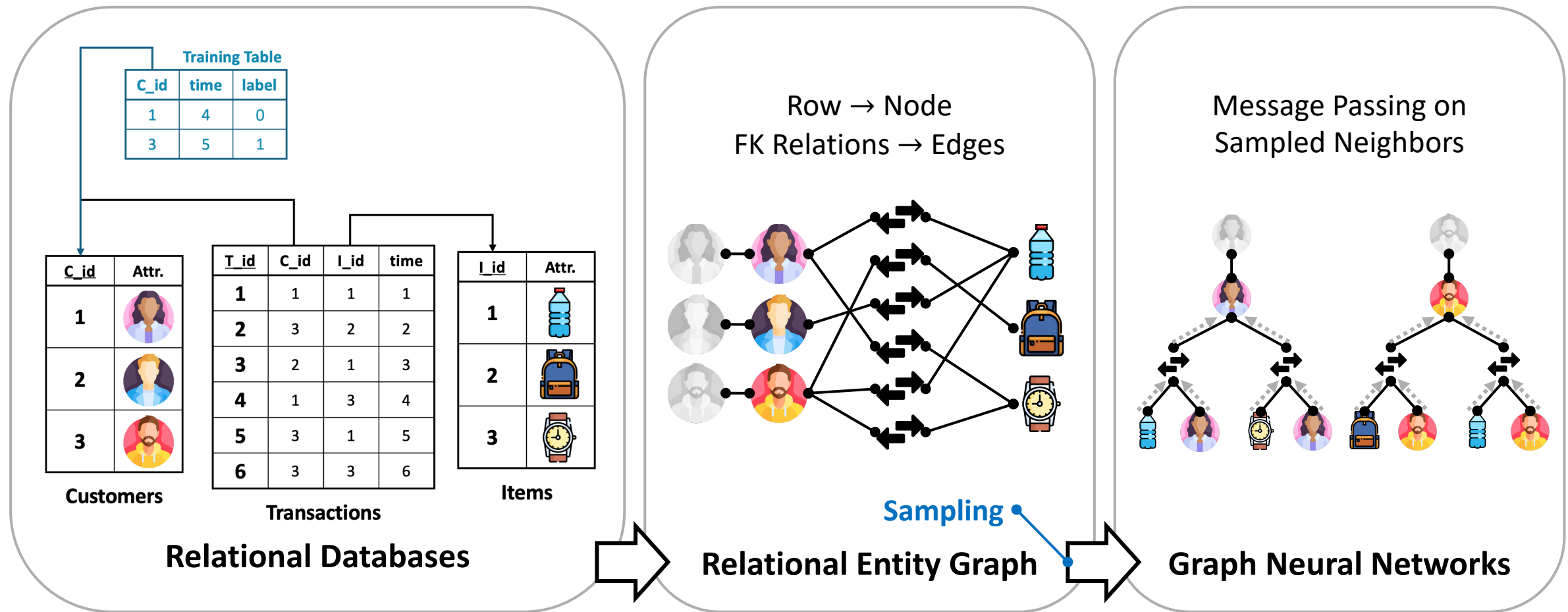
Matthias Fey<sup>\* 1</sup> Weihua Hu<sup>\* 1</sup> Kexin Huang<sup>\* 2</sup> Jan Eric Lenssen<sup>\* 1 3</sup> Rishabh Ranjan<sup>\* 2</sup> Joshua Robinson<sup>\* 2</sup>  
Rex Ying<sup>4</sup> Jiaxuan You<sup>5</sup> Jure Leskovec<sup>2 1</sup>

<sup>1</sup>Kumo.AI <sup>2</sup>Stanford University <sup>3</sup>Max Planck Institute for Informatics  
<sup>4</sup>Yale University <sup>5</sup>University of Illinois at Urbana-Champaign

ICML 2024 Position Paper

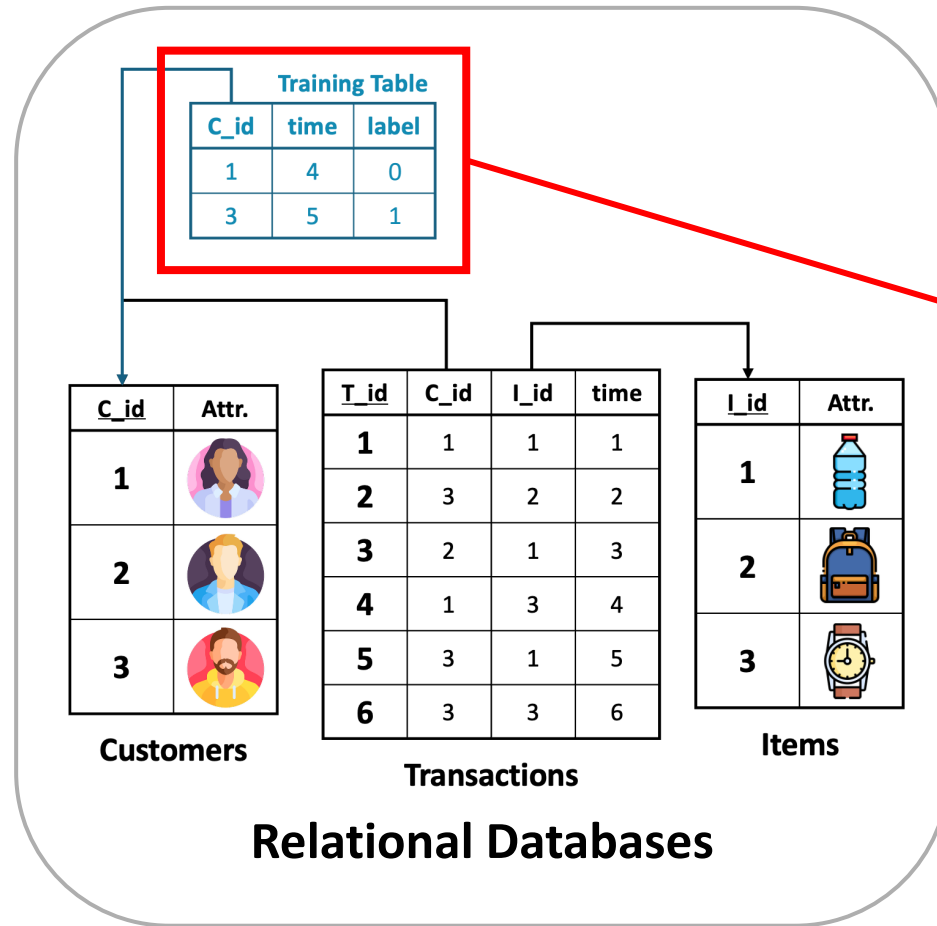
# Key Idea

- Represent Relational Databases as a temporal, heterogeneous Relational Entity Graph, and apply GNNs

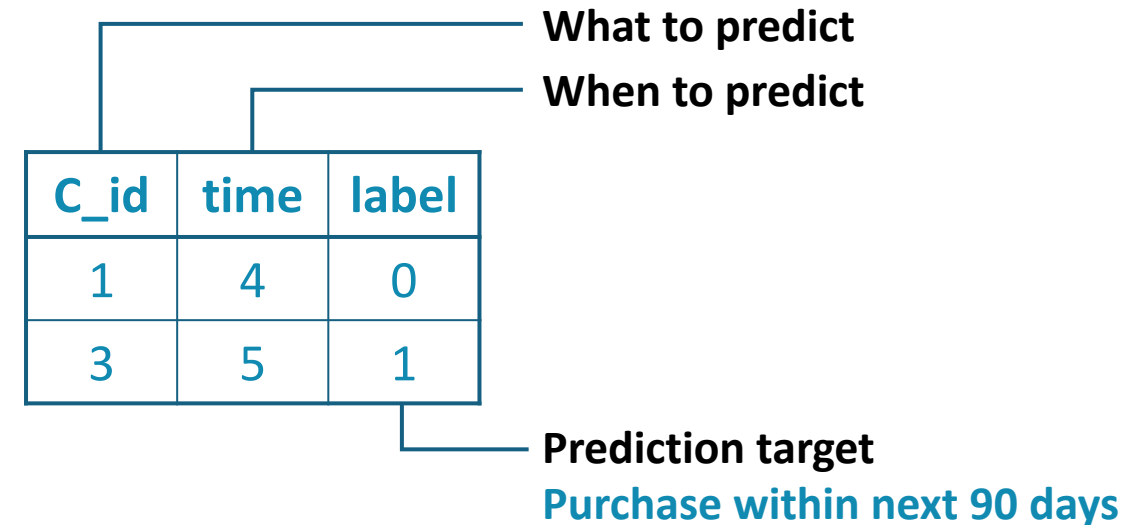


# How RDL Works – Task Specification

- (1) Given a task, a **training table** is automatically generated from historical RDB data



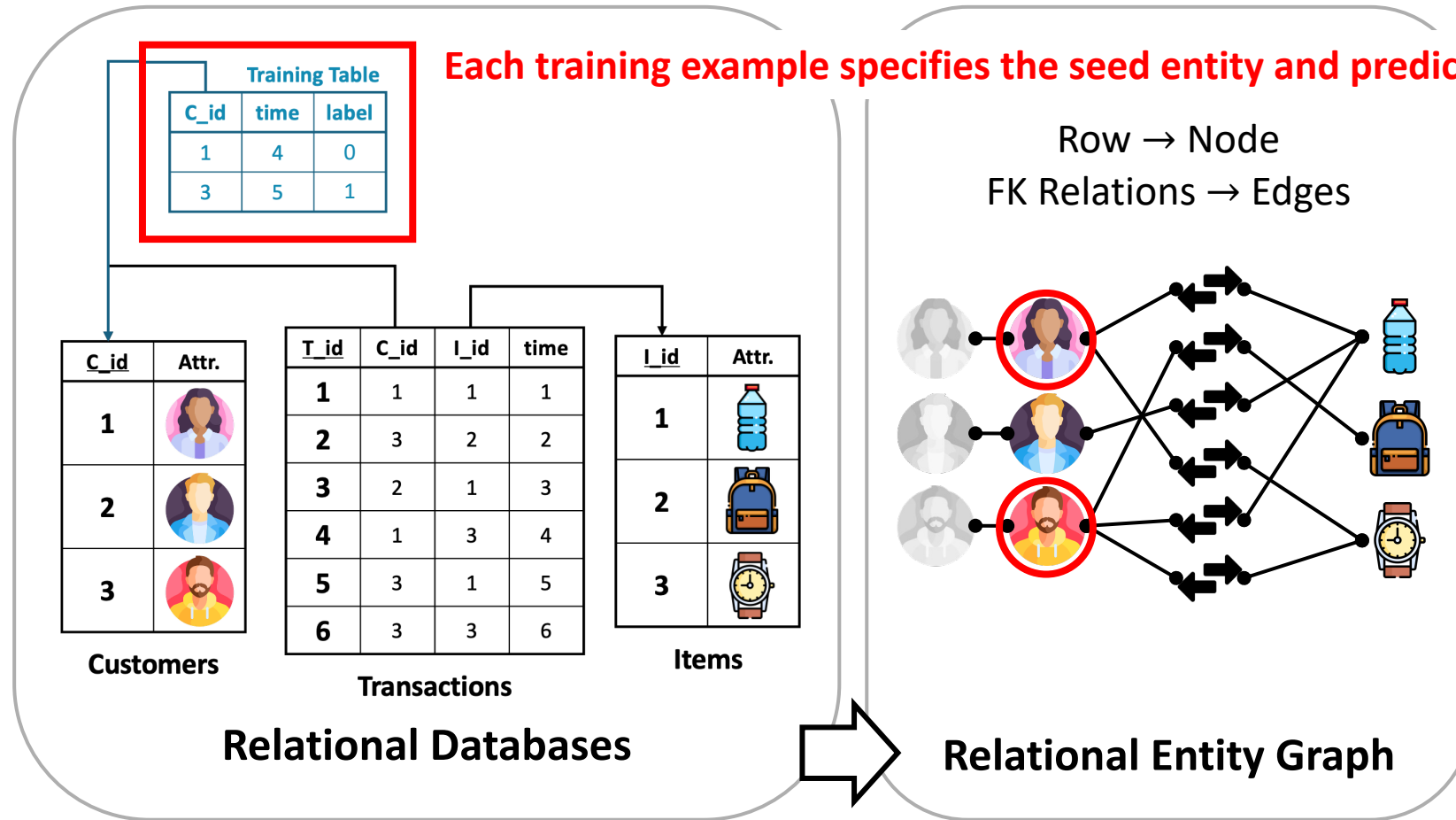
“Will each customer make a purchase within the next 90 days?”



Does not contain input data

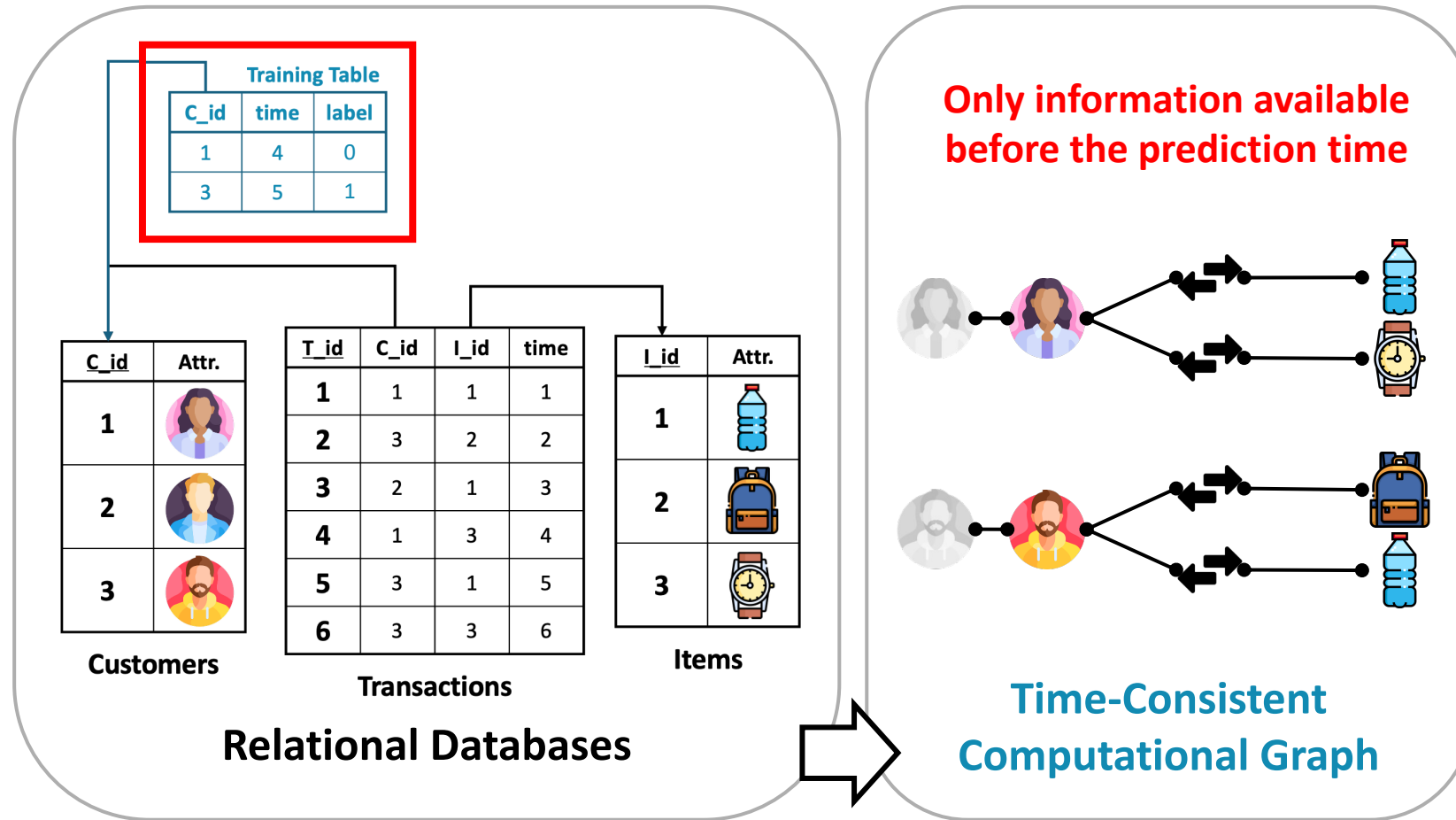
# How RDL Works – Graph Computation

- A **time-consistent computational graph** is sampled for each training example



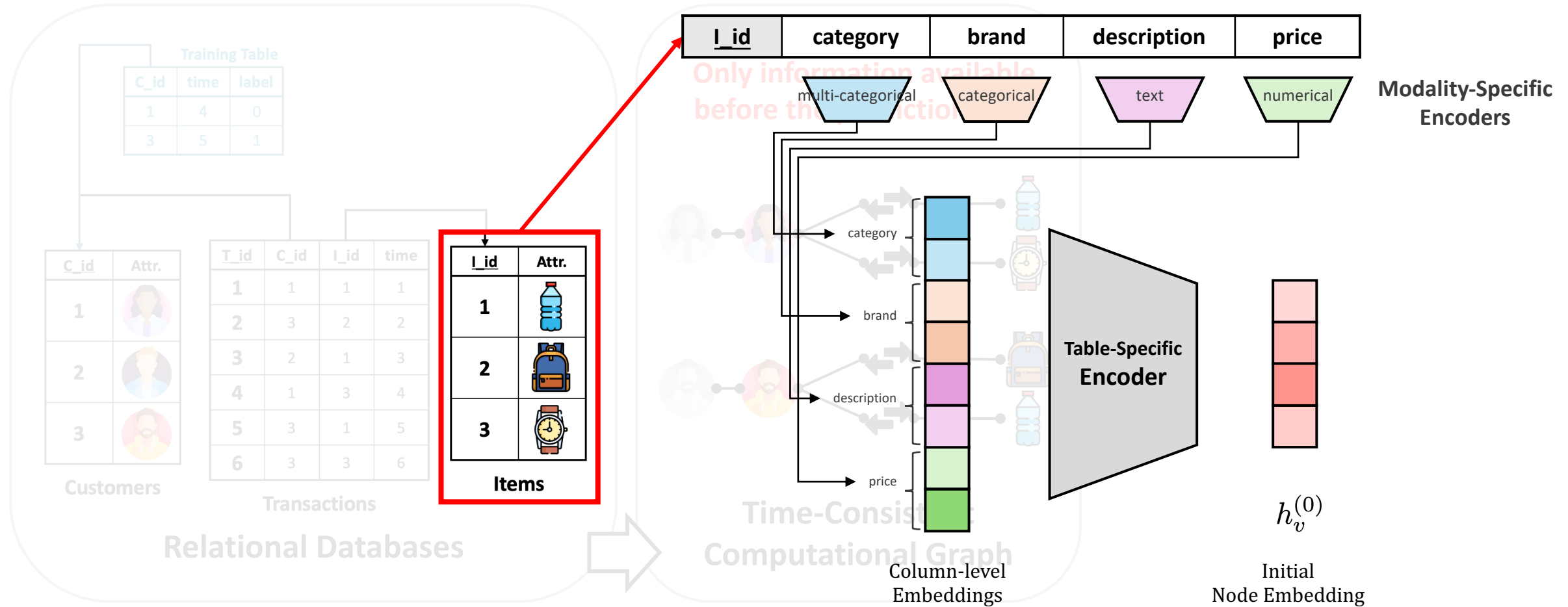
# How RDL Works – Graph Computation

- A **time-consistent computational graph** is sampled for each training example



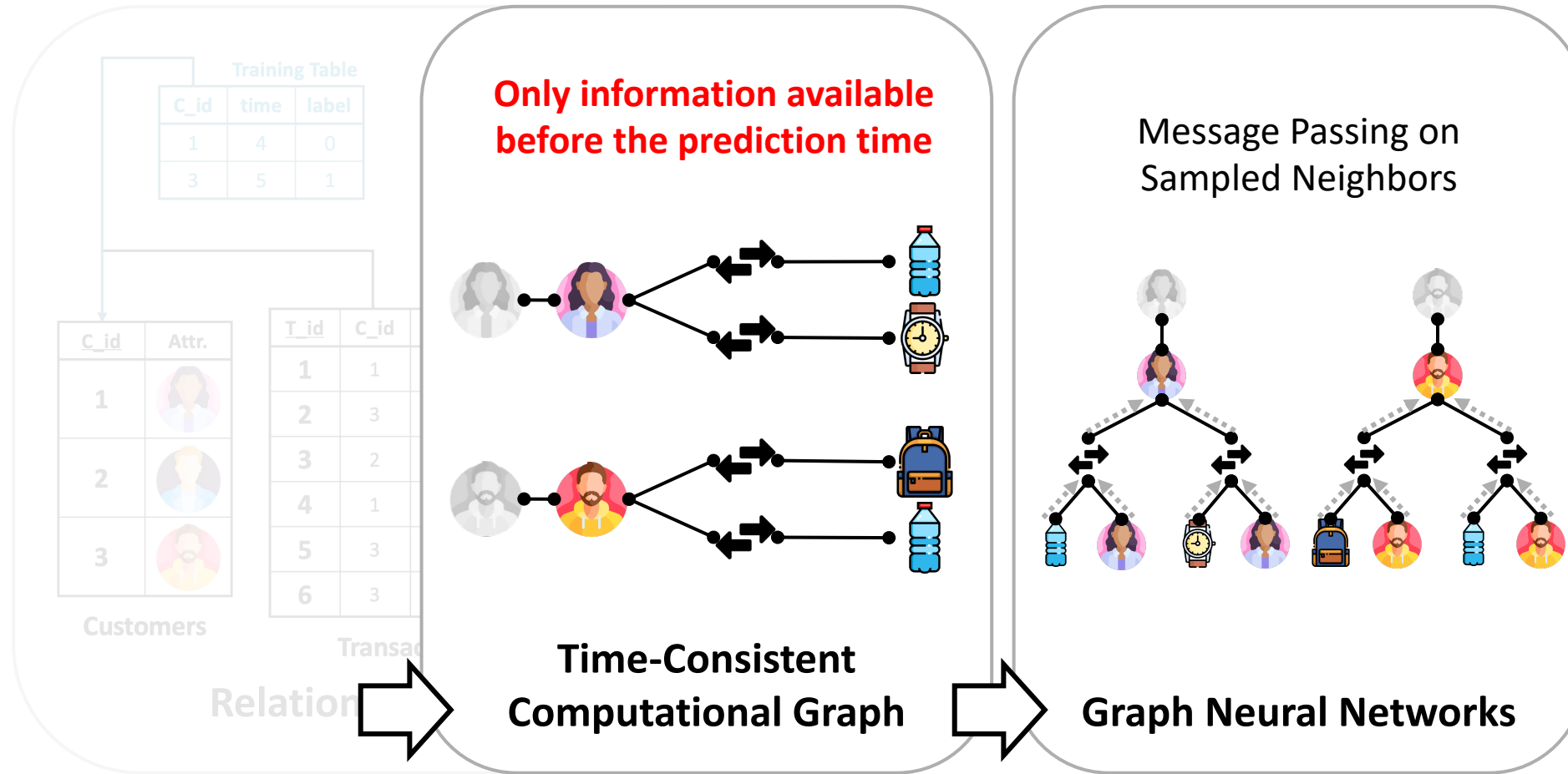
# How RDL Works – Initial Representation

## ■ (2) Each row's attributes are encoded as node features



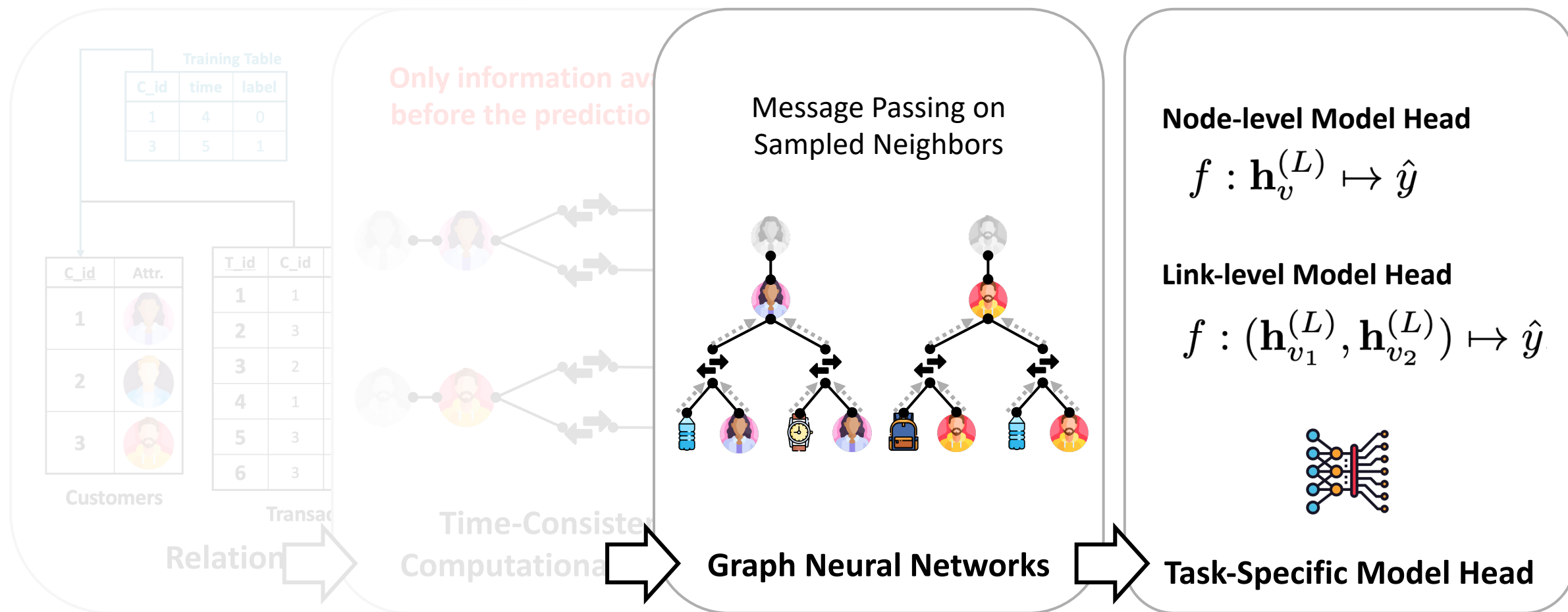
# How RDL Works – GNN Message Passing

## ■ (3) Node representations are learned through GNN message passing



# How RDL Works – Prediction & Learning

- (4) A task-specific model head produces predictions, and errors backpropagated

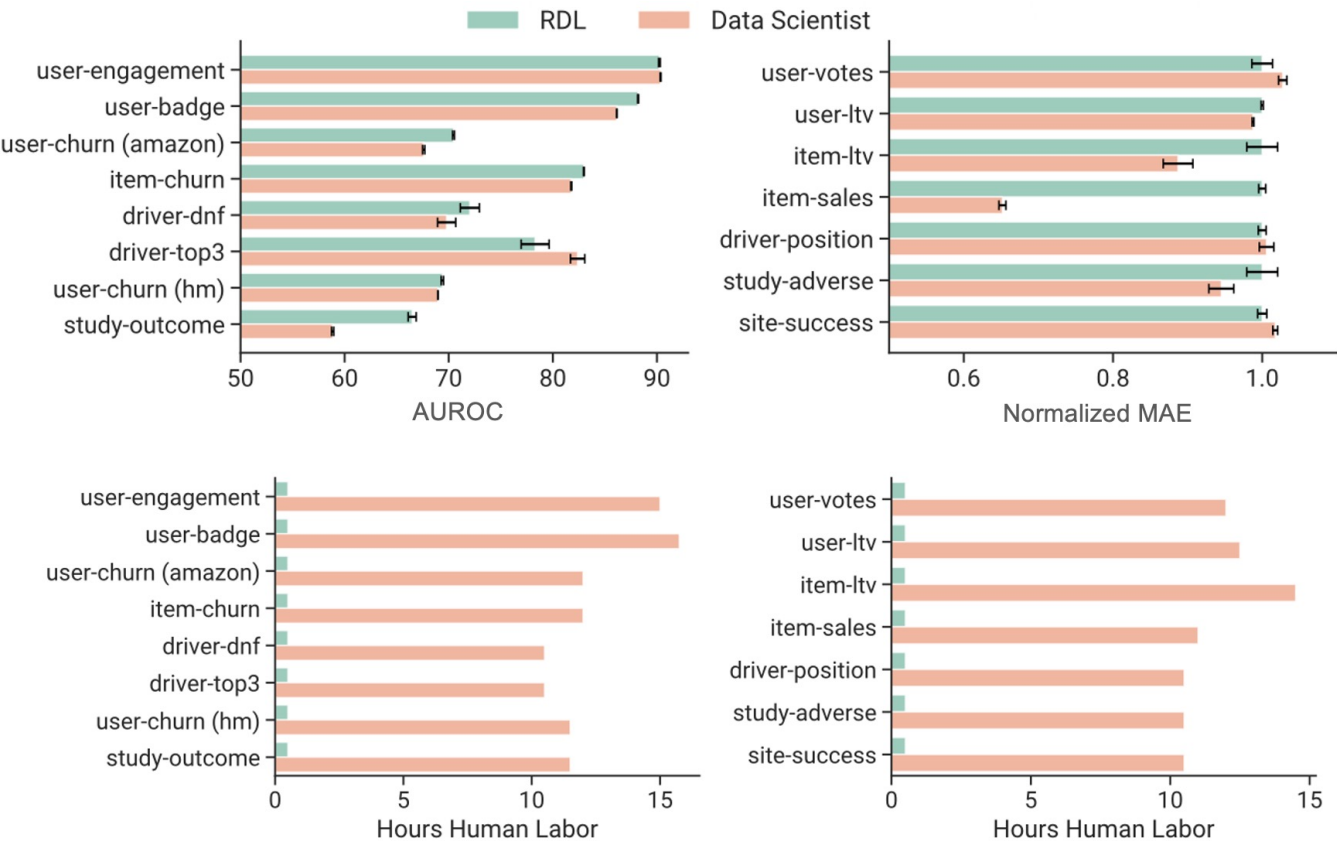


■ RDL models consistently outperform tabular baselines across diverse relational prediction tasks

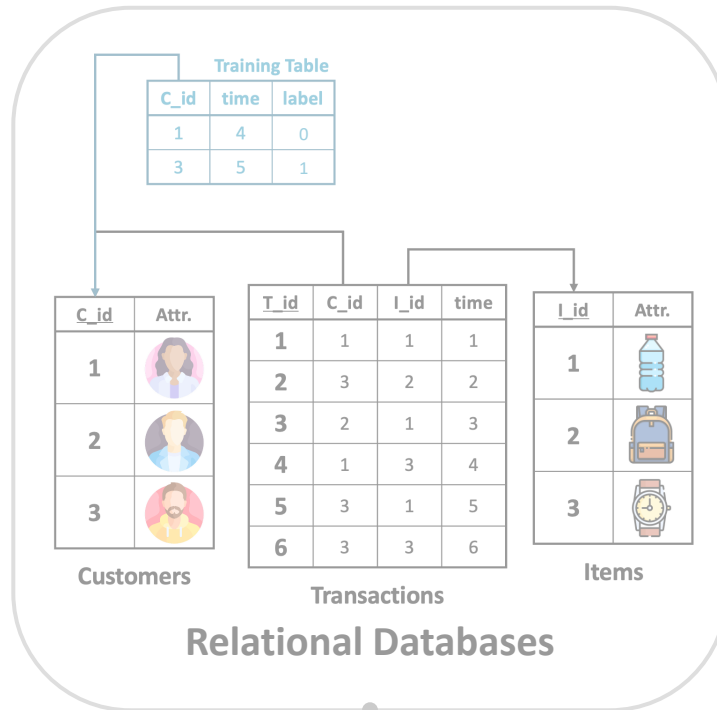
| Dataset    | Task            | Split | LightGBM     | RDL          | Rel. Gain of RDL |
|------------|-----------------|-------|--------------|--------------|------------------|
| rel-amazon | user-churn      | Val   | 52.05        | <b>70.45</b> | 35.35 %          |
|            |                 | Test  | 52.22        | <b>70.42</b> | 34.86 %          |
|            | item-churn      | Val   | 62.39        | <b>82.39</b> | 32.06 %          |
|            |                 | Test  | 62.54        | <b>82.81</b> | 32.40 %          |
| rel-avito  | user-visits     | Val   | 53.31        | <b>69.65</b> | 30.66 %          |
|            |                 | Test  | 53.05        | <b>66.20</b> | 24.78 %          |
|            | user-clicks     | Val   | 55.63        | <b>64.73</b> | 16.35 %          |
|            |                 | Test  | 53.60        | <b>65.90</b> | 22.96 %          |
| rel-event  | user-repeat     | Val   | 67.76        | <b>71.25</b> | 5.15 %           |
|            |                 | Test  | 68.04        | <b>76.89</b> | 13.02 %          |
|            | user-ignore     | Val   | 87.96        | <b>91.70</b> | 4.25 %           |
|            |                 | Test  | 79.93        | <b>81.62</b> | 2.12 %           |
| rel-fl     | driver-dnf      | Val   | 68.42        | <b>71.36</b> | 4.31 %           |
|            |                 | Test  | 68.56        | <b>72.62</b> | 5.93 %           |
|            | driver-top3     | Val   | 67.76        | <b>77.64</b> | 14.57 %          |
|            |                 | Test  | 73.92        | <b>75.54</b> | 2.20 %           |
| rel-hm     | user-churn      | Val   | 56.05        | <b>70.42</b> | 25.63 %          |
|            |                 | Test  | 55.21        | <b>69.88</b> | 26.59 %          |
| rel-stack  | user-engagement | Val   | 65.12        | <b>90.21</b> | 38.53 %          |
|            |                 | Test  | 63.39        | <b>90.59</b> | 42.91 %          |
|            | user-badge      | Val   | 65.39        | <b>89.86</b> | 37.43 %          |
|            |                 | Test  | 63.43        | <b>88.86</b> | 40.08 %          |
| rel-trial  | study-outcome   | Val   | <b>68.30</b> | 68.18        | −0.19 %          |
|            |                 | Test  | <b>70.09</b> | 68.60        | −2.13 %          |
| Average    |                 | Val   | 64.18        | <b>76.49</b> | 20.34 %          |
|            |                 | Test  | 63.66        | <b>75.83</b> | 20.48 %          |

Classification (ROC-AUC (%), ↑)

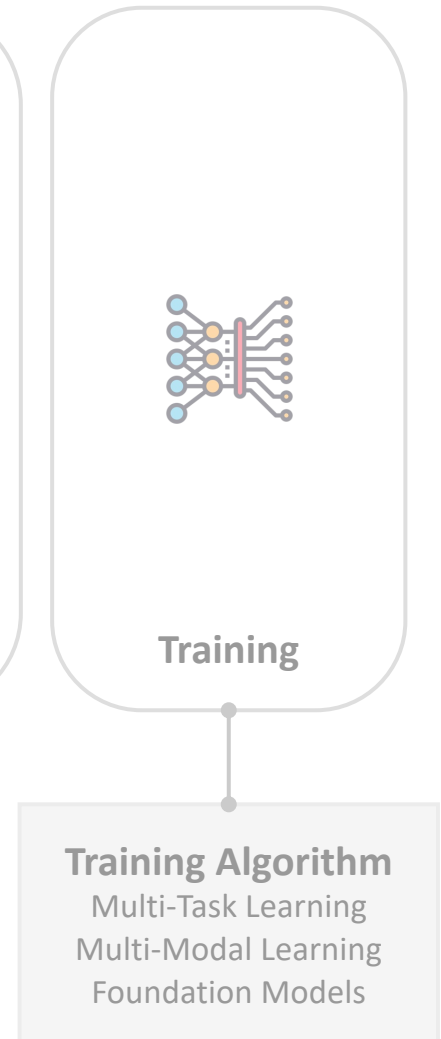
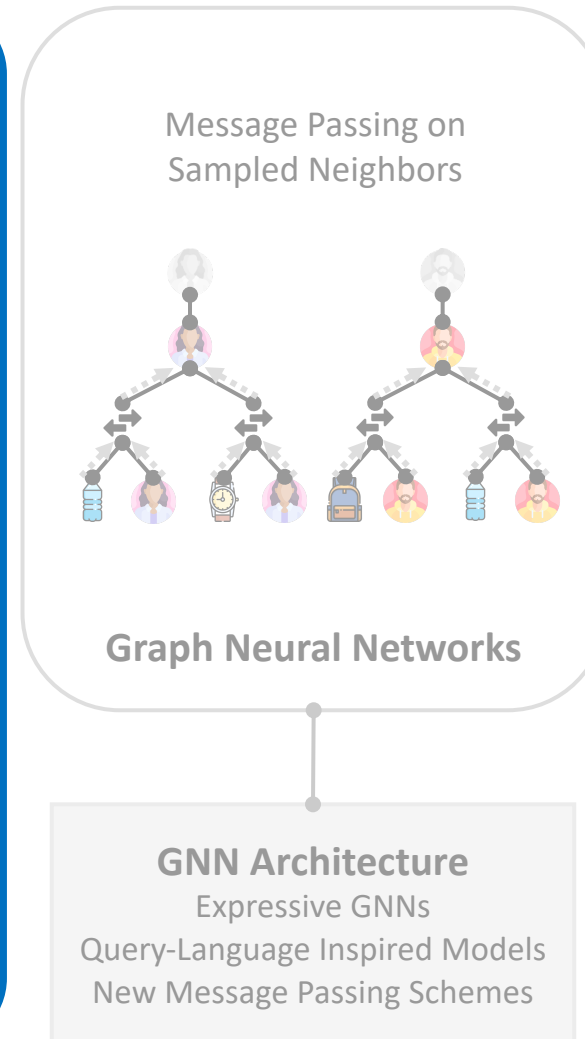
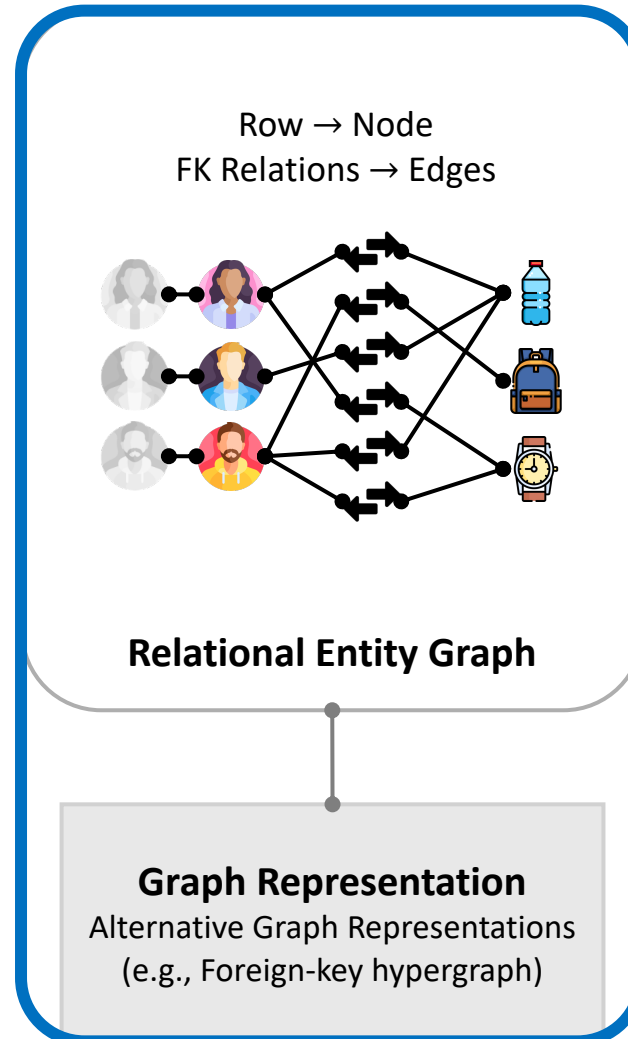
■ RDL vs. DataScientist



# Future Directions



**Relational Databases**  
Scaling RDL

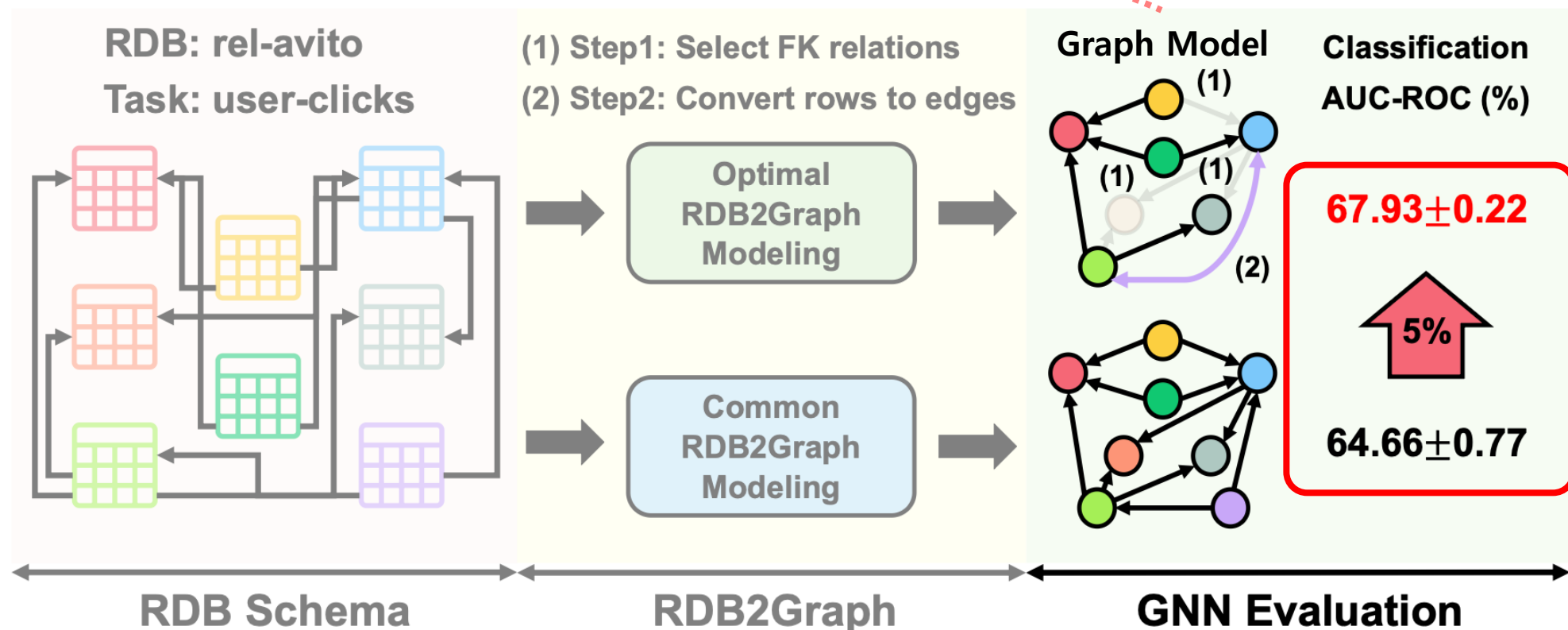


# Why Graph Modeling?

From RDB2G-Bench (NeurIPS 2026)

- Different graph representations from the same RDB can lead to substantially different predictive performance

Same RDB & Task, Different Graph Models



# Why Graph Modeling?

From RDB2G-Bench (NeurIPS 2026)



## RDB2G-Bench Datasets

Precomputed performance metrics

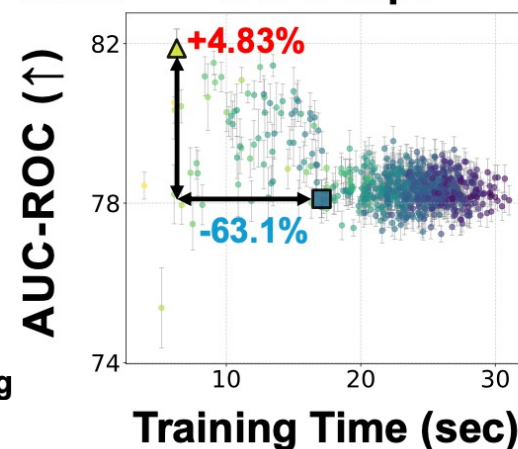
- Predictive performance (train/val/test)
- Runtime
- Parameter size

Setup

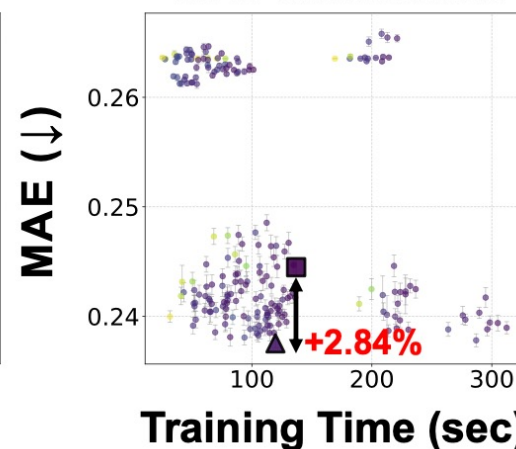
- Classification, Regression: HeteroSAGE
- Recommendation: ID-GNN

| RDB       | Task Name               | Type           | # Tables | # Graph Models | Performance Statistics |           |       |
|-----------|-------------------------|----------------|----------|----------------|------------------------|-----------|-------|
|           |                         |                |          |                | Best                   | AR2N [29] | Worst |
| rel-avito | user-clicks (UC)        | classification | 8        | 944            | 67.93                  | 64.66     | 60.89 |
|           | user-visits (UV)        | classification |          | 944            | 66.33                  | 65.97     | 59.83 |
|           | ad-ctr (AC)             | regression     |          | 1304           | 0.039                  | 0.040     | 0.044 |
|           | user-ad-visit (UAV)     | recommendation |          | 909            | 3.682                  | 3.661     | 0.159 |
| rel-event | user-repeat (UR)        | classification | 5        | 214            | 82.29                  | 77.65     | 63.96 |
|           | user-ignore (UI)        | classification |          | 214            | 82.82                  | 82.22     | 74.29 |
|           | user-attendance (UA)    | regression     |          | 214            | 0.237                  | 0.244     | 0.266 |
| rel-f1    | driver-dnf (DD)         | classification | 9        | 722            | 74.56                  | 73.14     | 67.40 |
|           | driver-top3 (DT)        | classification |          | 722            | 81.88                  | 78.11     | 75.37 |
|           | driver-position (DP)    | regression     |          | 722            | 3.831                  | 3.913     | 4.171 |
| rel-stack | post-post-related (PPL) | recommendation | 7        | 7979           | 12.04                  | 10.82     | 0.006 |
| rel-trial | study-outcome (SO)      | classification | 15       | 36863          | 70.91                  | 68.09     | 62.85 |

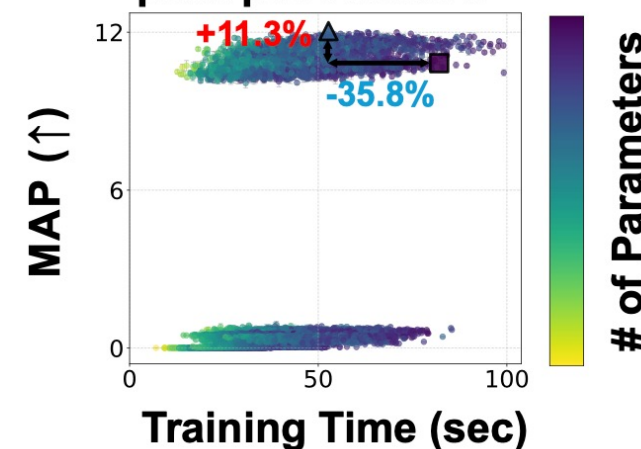
Task: driver-top3



Task: user-attendance



Task: post-post-related



□ : All-Rows-to-Nodes (AR2N) Modeling

△ : Best-performing Graph Model

# What Makes a Desired Graph for Relational Deep Learning?

Yao Cheng<sup>1</sup> Siqiang Luo<sup>1</sup>

<sup>1</sup>Nanyang Technological University

ICML 2026

# Motivation

## ■ Fundamental Mismatch

- RDBs are engineered for *data integrity* and *storage efficiency*, **not for representation learning**

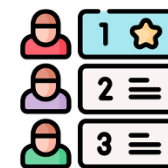
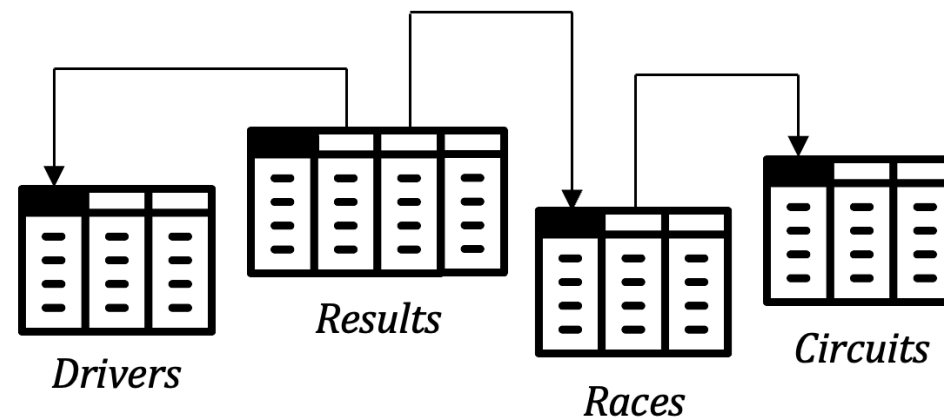
↳ Decomposing information across multiple normalized tables

“*[driver] finished [position] in [race] at [circuit].*”

| driver | position | race | circuit |
|--------|----------|------|---------|
|        |          |      |         |
|        |          |      |         |
|        |          |      |         |



→  
In RDB,

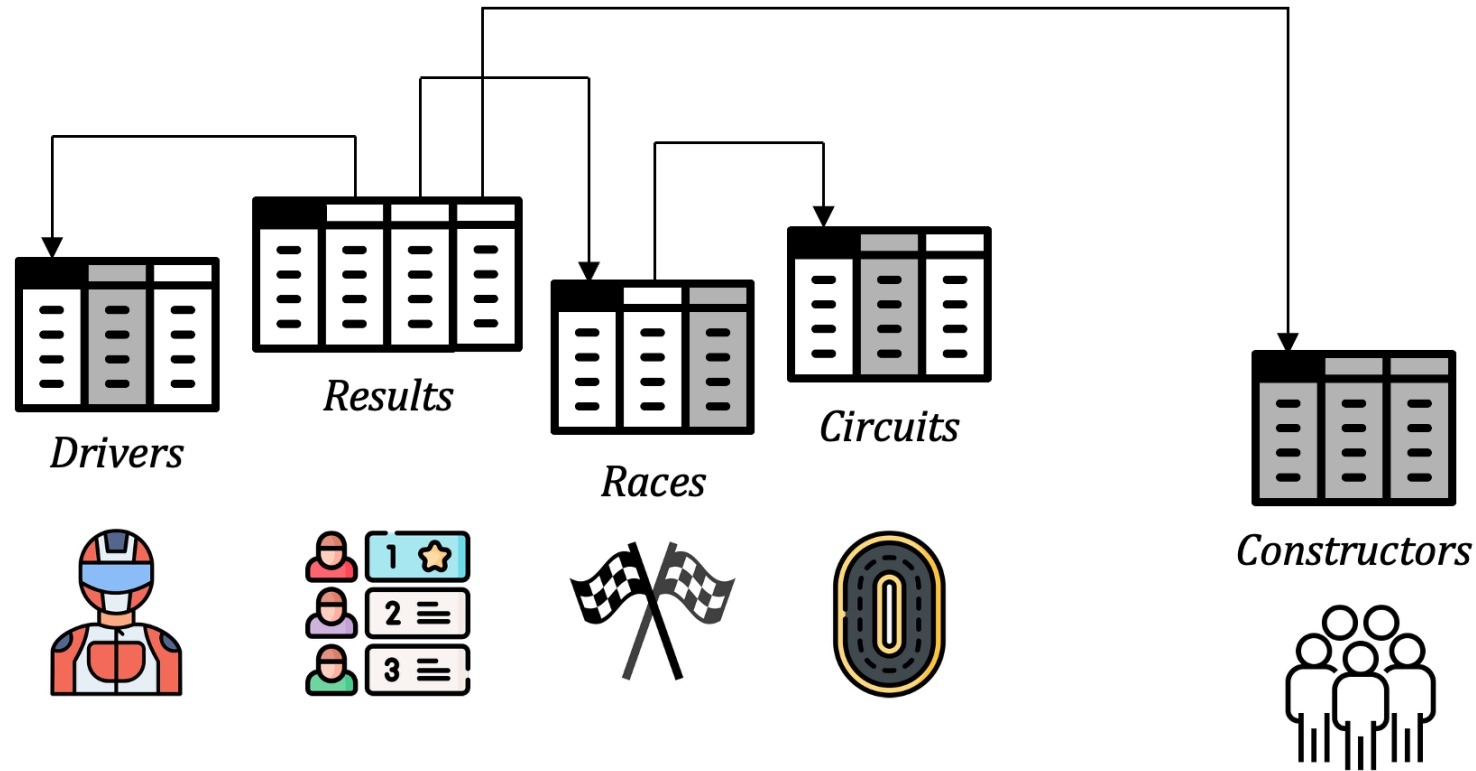


⇒ Schema-derived Graph  
(Row = Node , FK links = Edge)

# Motivation

## ■ Schema-derived Graph's Limitation 1: Information Overload

- Some columns and tables are **less relevant** to the prediction target



# Motivation

## ■ Schema-derived Graph's Limitation 2: **Semantic Fragmentation**

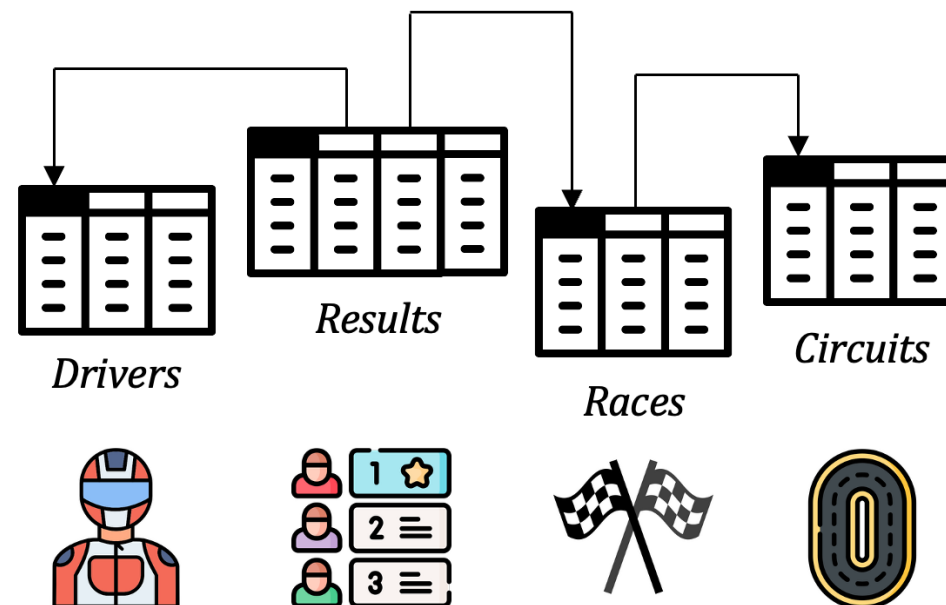
- Decomposing relationships into chains of FK links → **Traverse long paths** to recover a dependency

*"[driver] finished [position] in [race] at [circuit]."*

| driver | position | race | circuit |
|--------|----------|------|---------|
|        |          |      |         |
|        |          |      |         |
|        |          |      |         |



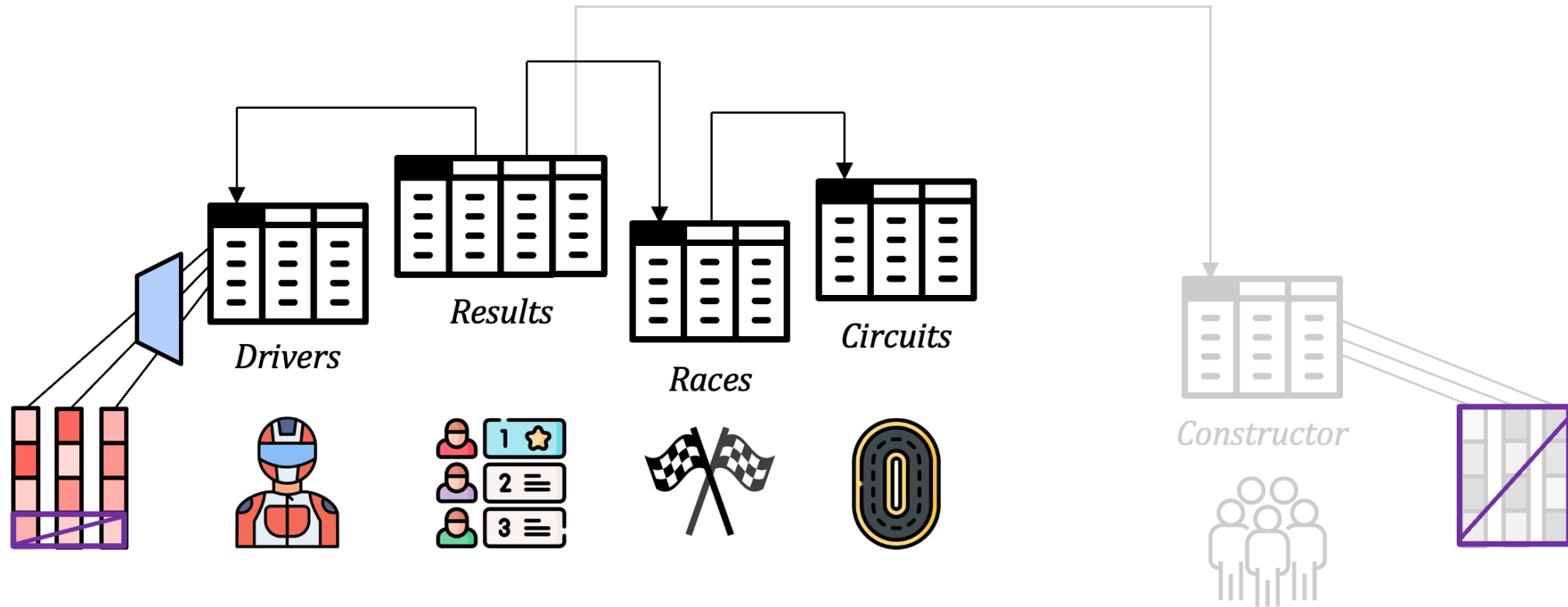
→  
In RDB,



Task-relevant concepts are separated by multiple FK hops

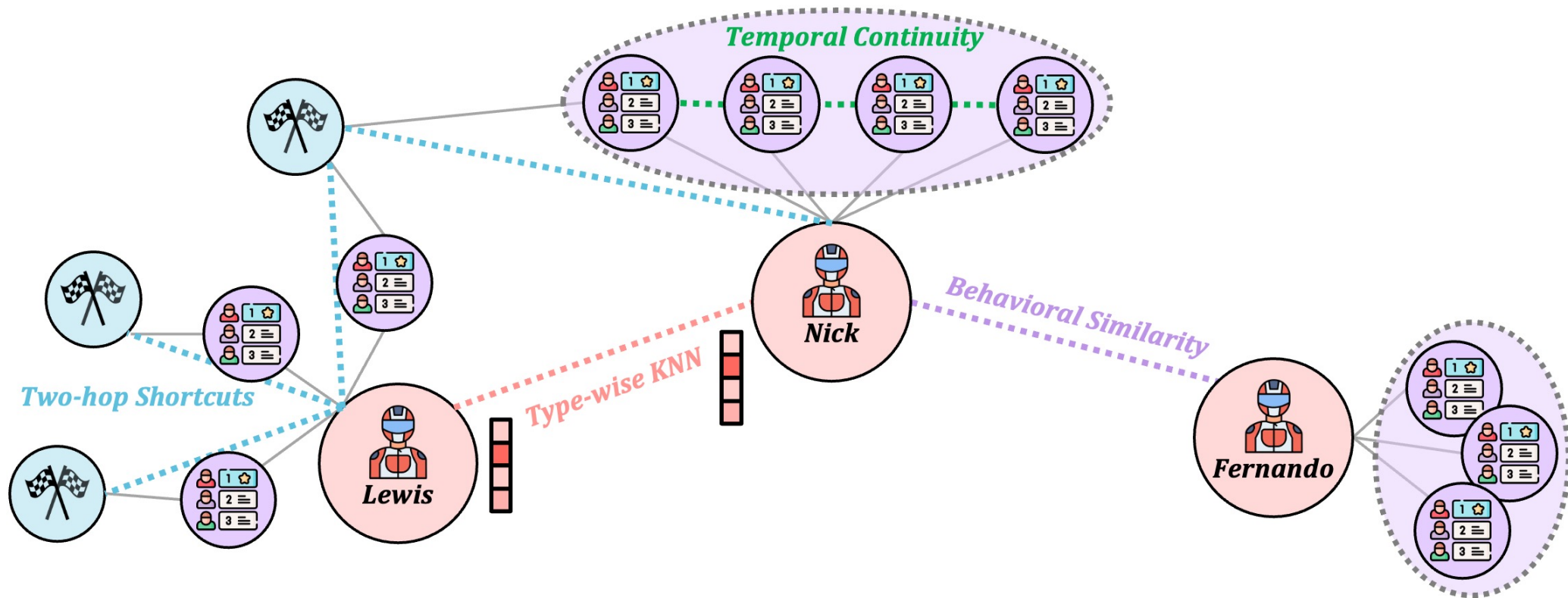
# Key Idea

- Learn a task-dependent graph  
by filtering irrelevant information and **injecting** useful structure



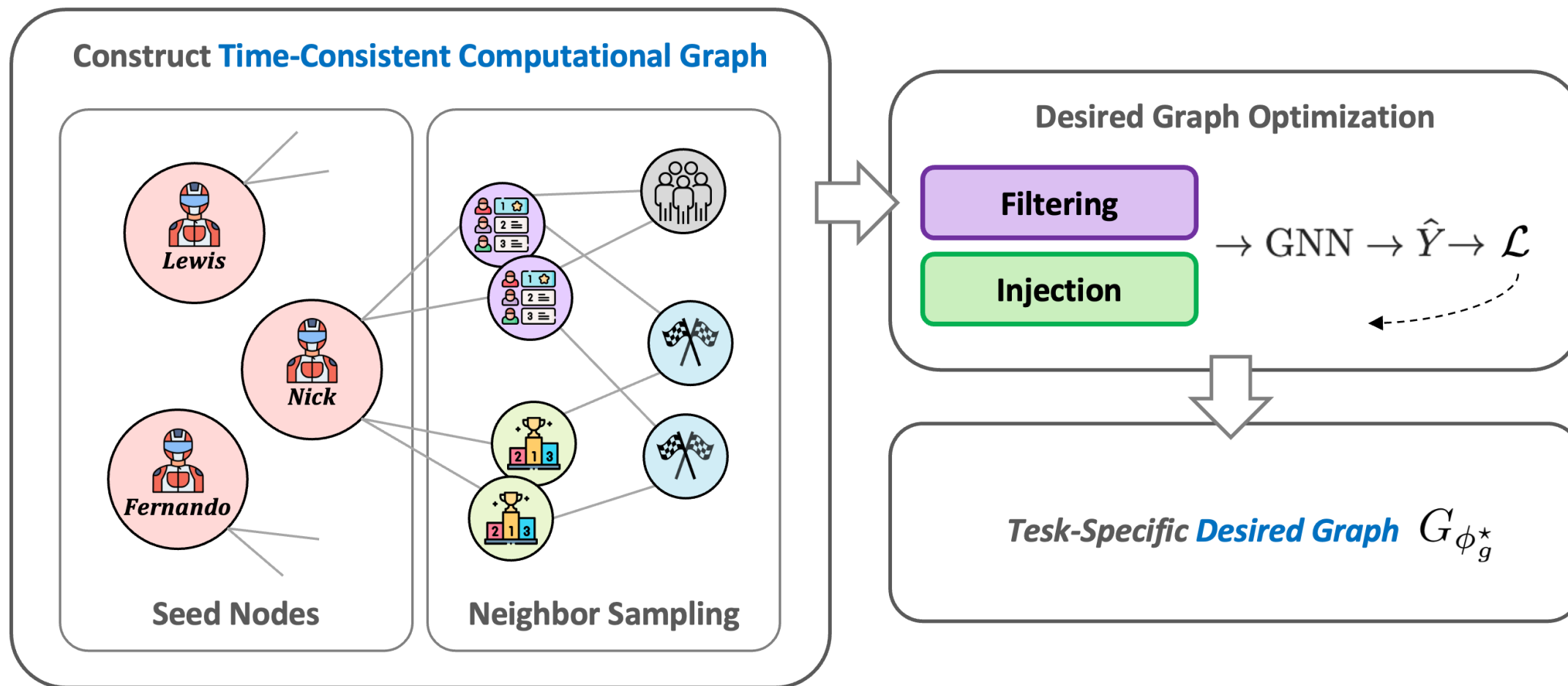
# Key Idea

- Learn a task-dependent graph  
by **filtering** irrelevant information and **injecting** useful structure



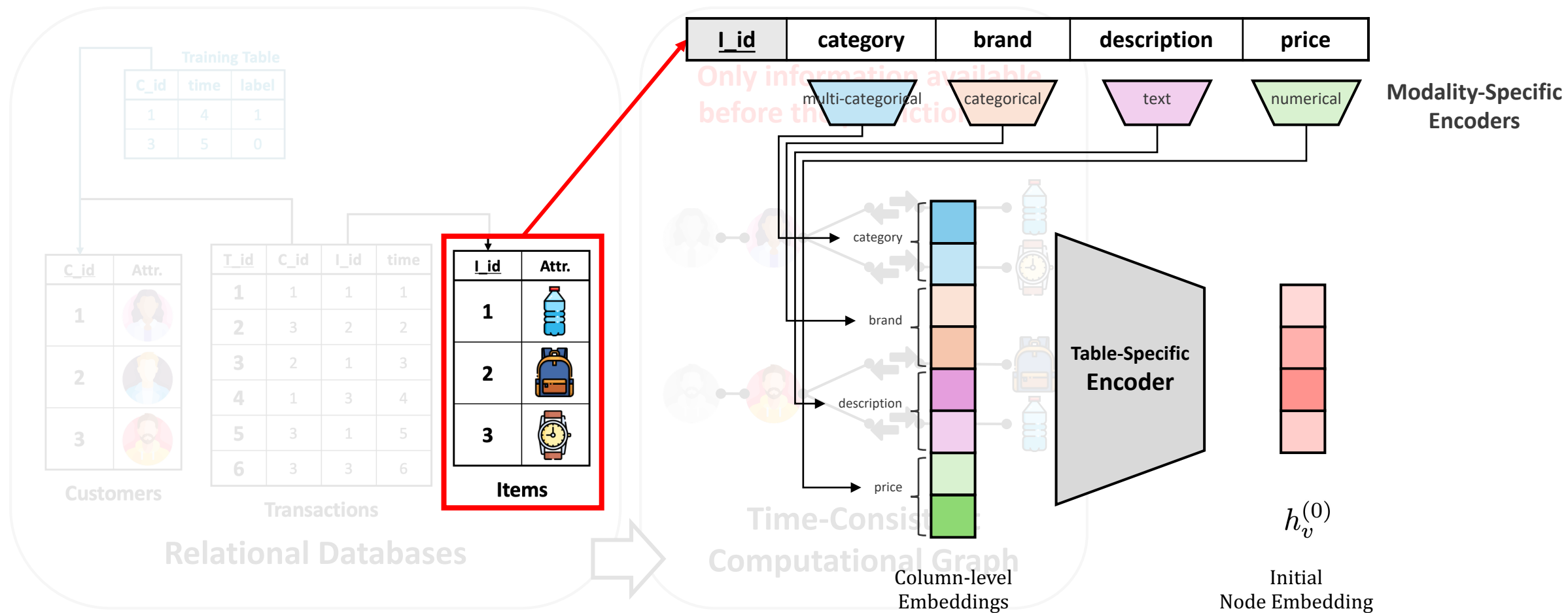
# Method Overview

## ■ Model Operates on Time-Consistent Computational Graph



# How RDL Works – Initial Representation (Recap)

- (2) Each row's attributes are encoded as node features



# Method (1) Information Filtering

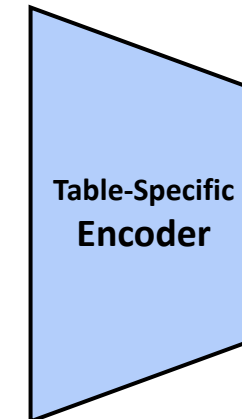
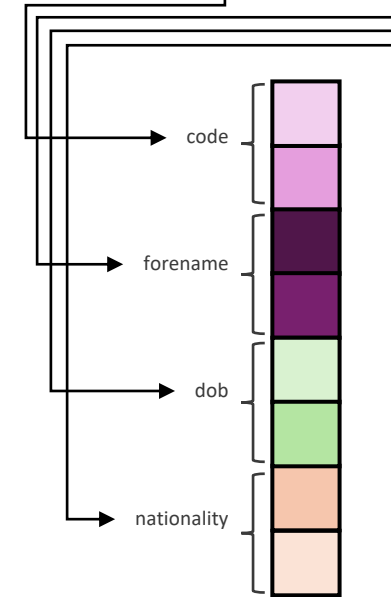
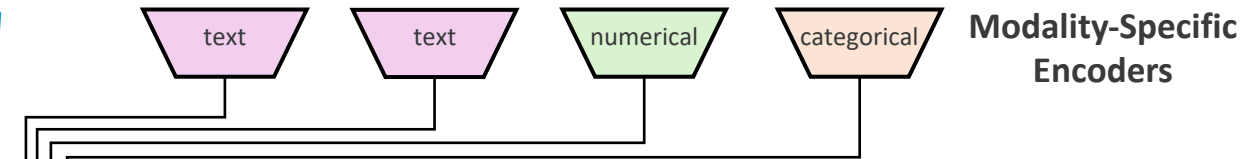
- Column information is mixed into a single node representation

Drivers

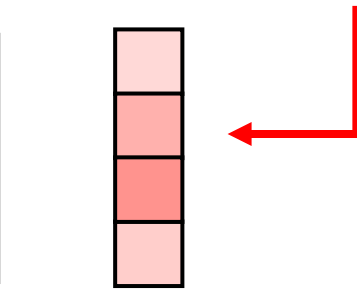


| driverId | code | forename | dob        | nationality                                                                               |
|----------|------|----------|------------|-------------------------------------------------------------------------------------------|
| 0        | HAM  | Lewis    | 1985-01-07 |  British |
| 1        | HEI  | Nick     | 1977-05-10 |  German  |
| 3        | ALO  | Fernando | 1981-07-29 |  Spanish |
| :        | :    | :        | :          | :                                                                                         |

| <u>driverId</u> | code | forename | dob | nationality |
|-----------------|------|----------|-----|-------------|
|-----------------|------|----------|-----|-------------|



Filtering after feature mixing

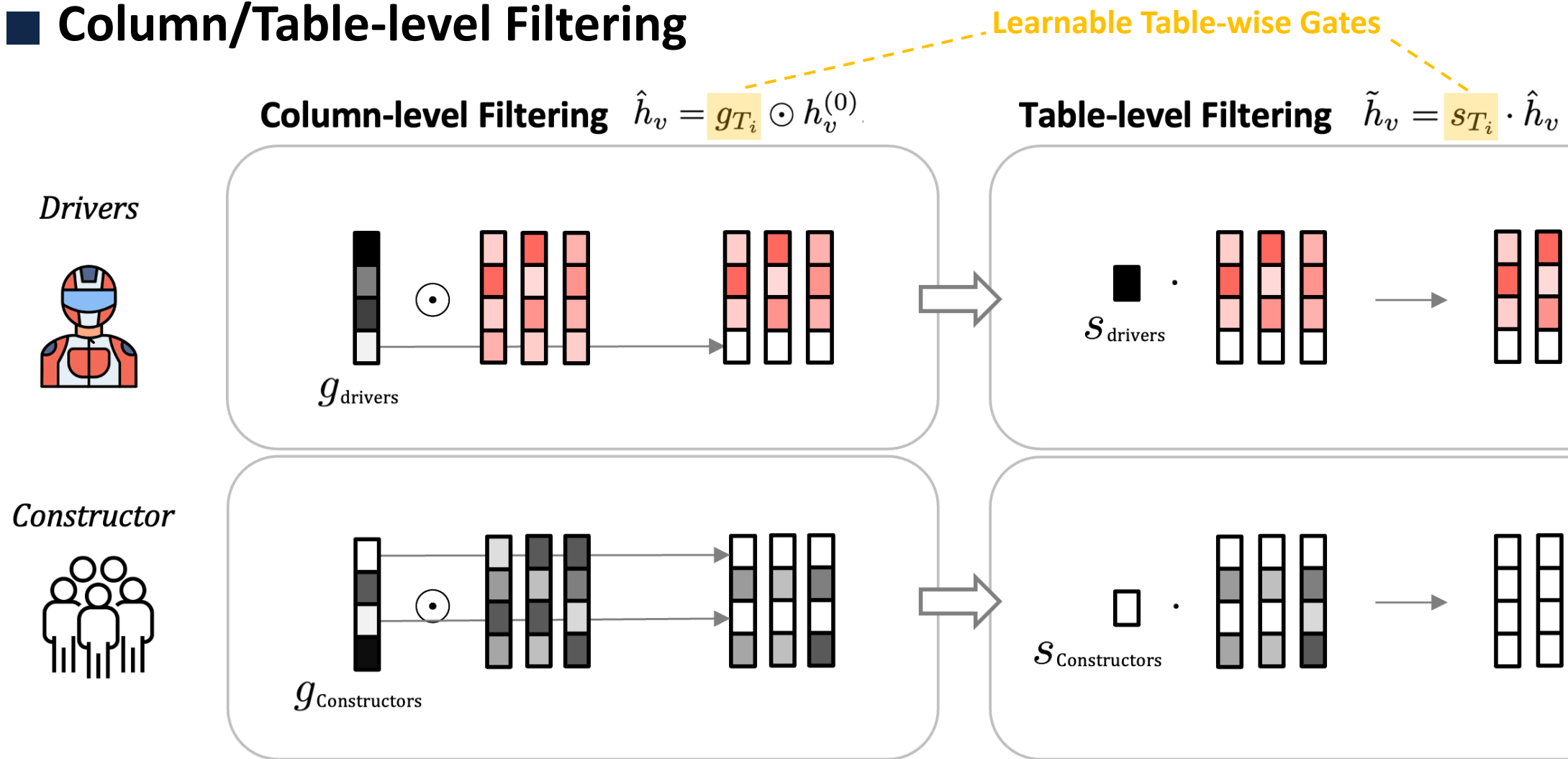


$h_v^{(0)}$

Initial Node Embedding

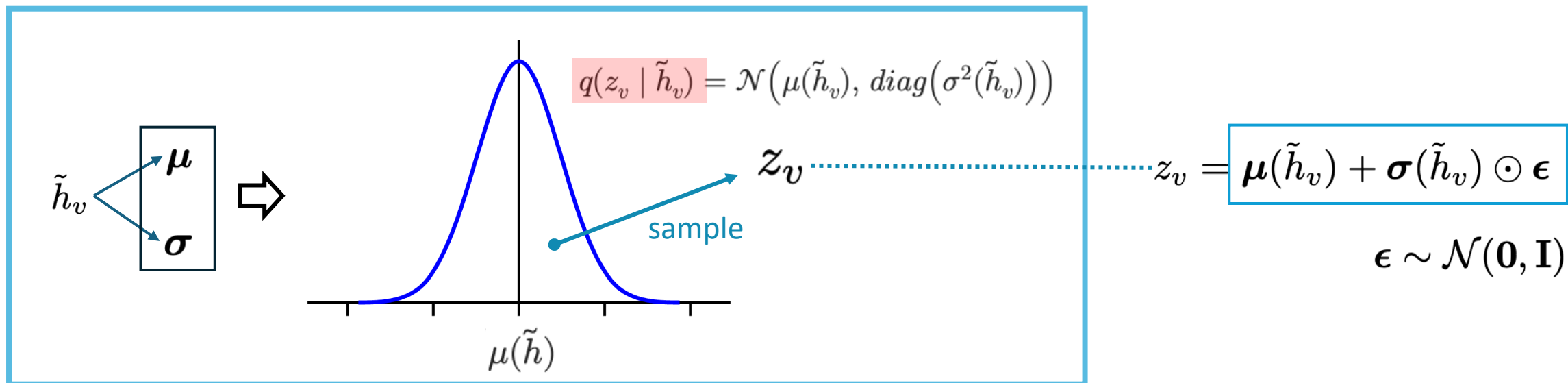
# Method (1) Information Filtering (Cont'd)

## Column/Table-level Filtering



# Method (1) Information Filtering (Cont'd)

## ■ Compress the Filtered Representation with VIB

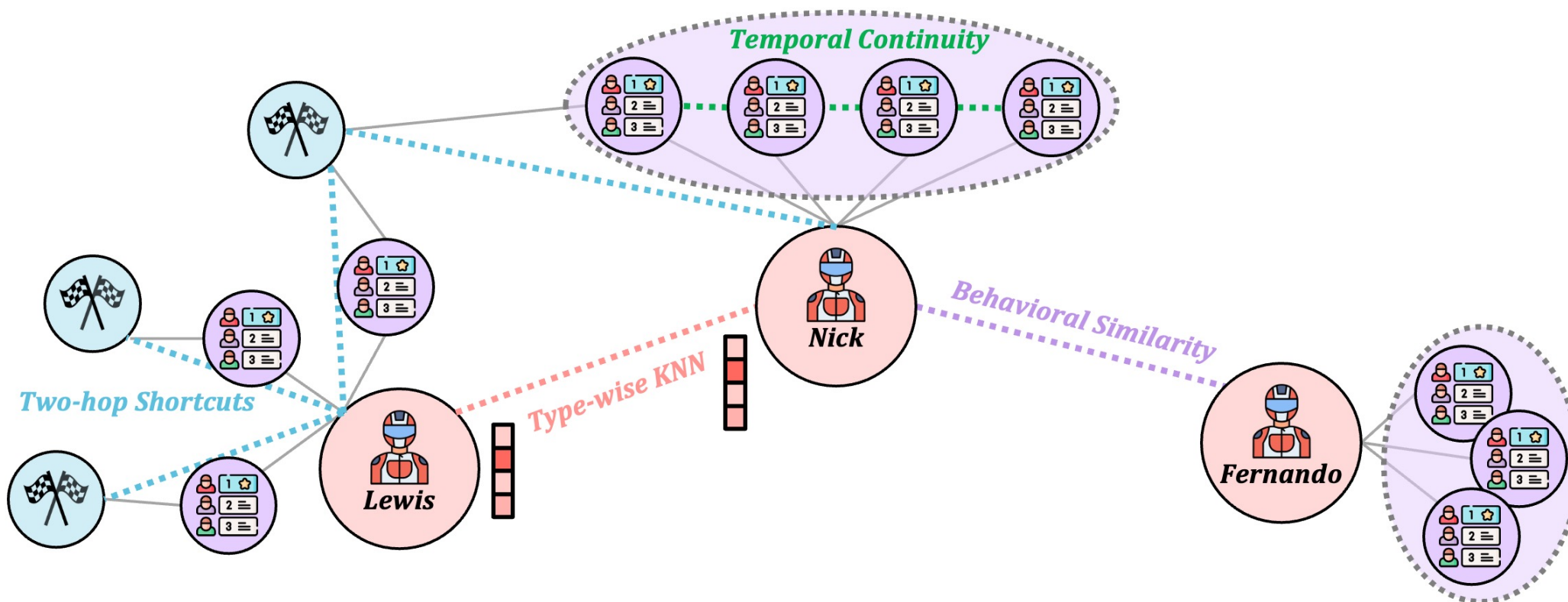


**VIB Regularization**  $\text{KL}(q(z_v | \tilde{h}_v) \| \mathcal{N}(\mathbf{0}, \mathbf{I}))$

**: Limits information passed from  $\tilde{h}_v$  to  $z_v$**   
**→ Compact latent representation**

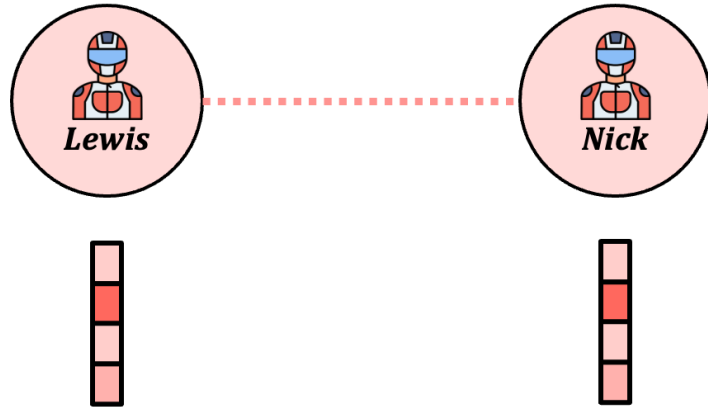
# Method (2) Structural Injection

■ 4 Strategies: Type-wise KNN, Behavioral Similarity, Two-hop Shortcuts, Temporal Continuity



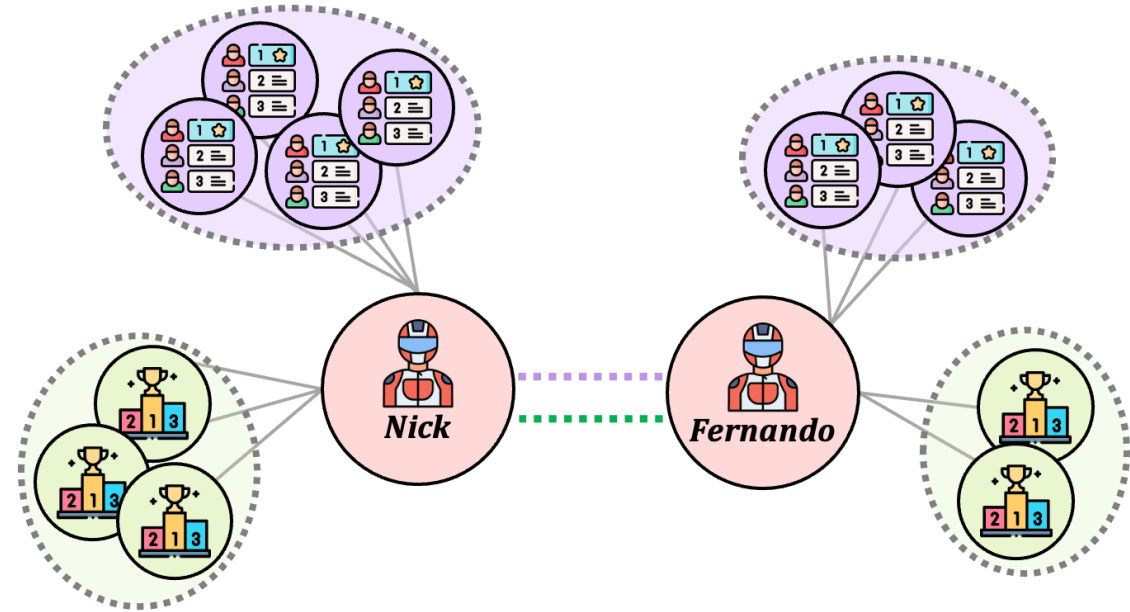
# Method (2) Structural Injection – 4 strategies

## ① Type-wise KNN (Homophily)



$$\mathcal{E}_{knn}^{(i)} = \{(u, v) \mid v \in \text{Top-}n_{v' \in T_i}(\text{sim}(h_u^{(0)}, h_{v'}^{(0)}))\}$$

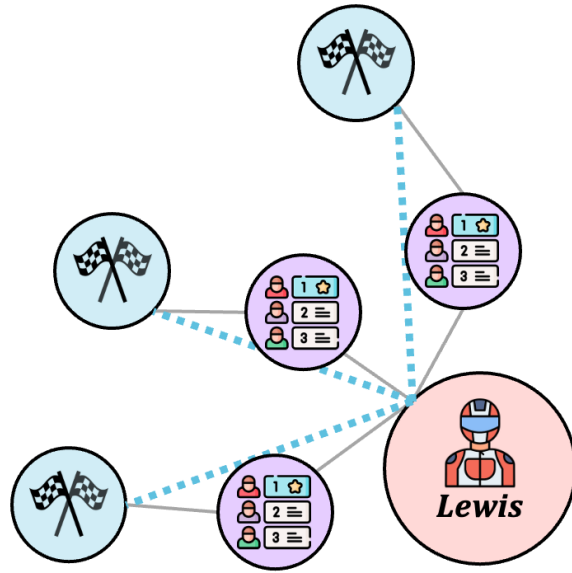
## ② Behavioral Similarity (Collaboration)



$$\mathcal{E}_{\text{sim}} = \{(v_{a_i}, v_{a_j}) \mid \text{sim}(f_{\text{agg}}(\mathcal{N}_B^r(v_{a_i})), f_{\text{agg}}(\mathcal{N}_B^r(v_{a_j}))) > \tau_{\text{sim}}\}$$

# Method (2) Structural Injection – 4 strategies

## ③ Two-hop Shortcuts (Topology)



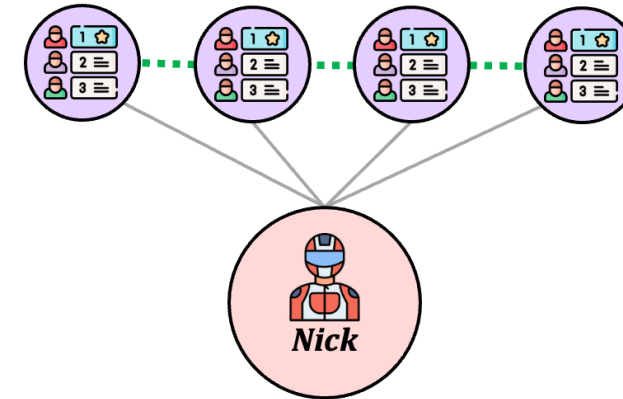
$$\mathcal{E}_{2hop} = \{(u_a, u_c) \mid \exists u_b \text{ s.t. } (u_a, u_b) \in \mathcal{E}_{r_1} \wedge (u_b, u_c) \in \mathcal{E}_{r_2}\}$$








## ④ Temporal Continuity (Causal)



$$E_{\text{time}} = \{(v_{t-1}, v_t) \mid t = 2, \dots, m\}$$

# Method (2) Structural Injection – Template-wise Gating

**Strategy:** General rule for injecting relations

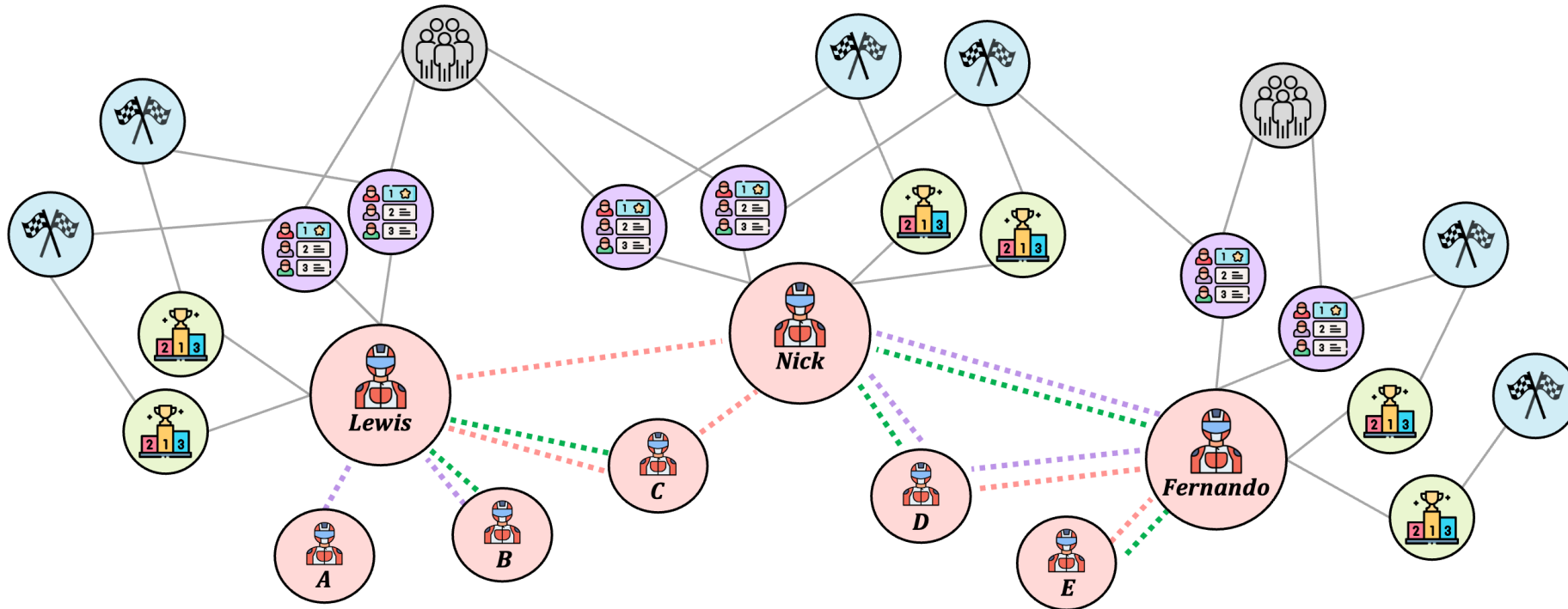
(e.g., Type-wise KNN, Behavioral Similarity, ...)

**Template  $k$ :** Type/relation-specific application of a strategy

(e.g., drivers KNN, drivers-results BS, drivers-standings BS, ...)

**Each colored relation = one template  $k$ ,**  $z_{v,k}^{(l)} = g_k \cdot \text{AGG}(z_u^{(l)} : u \in \mathcal{N}_k(v))$

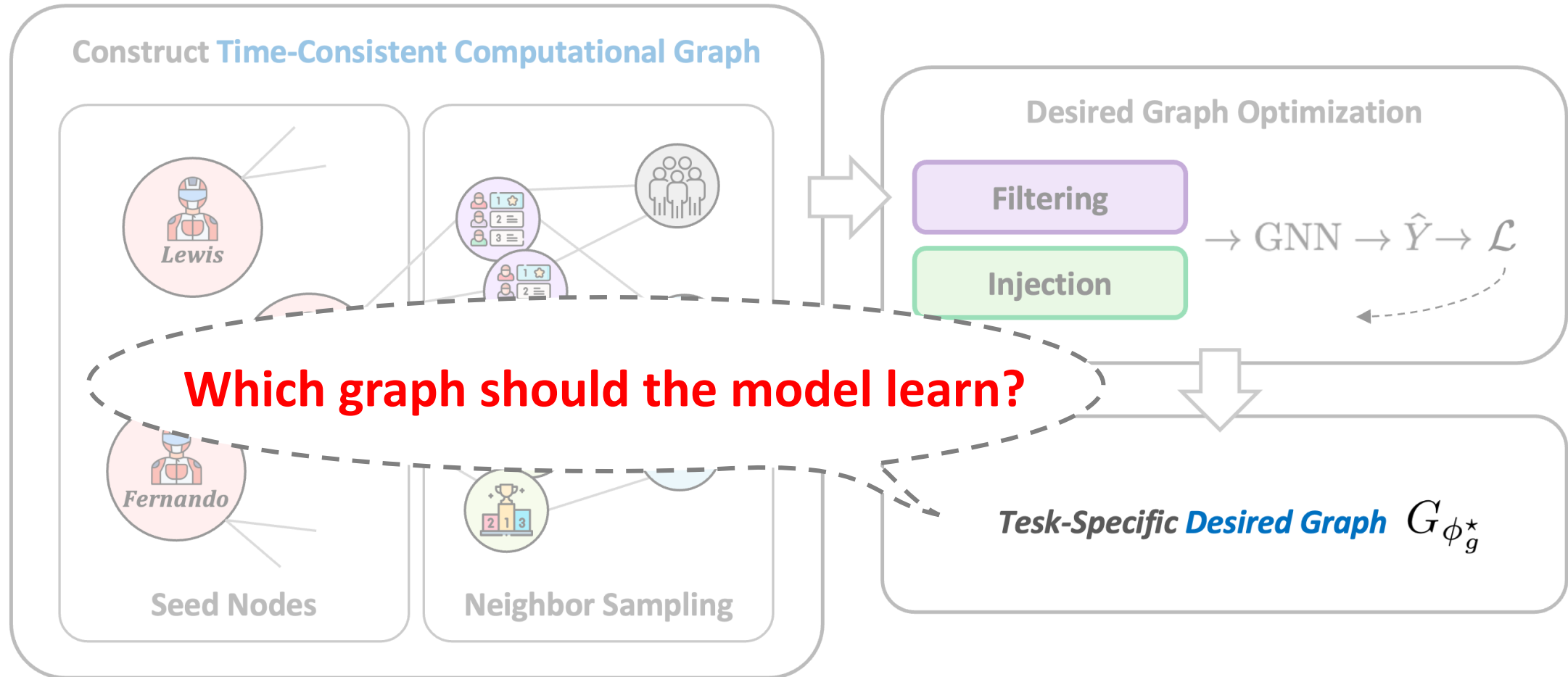
**Learnable Template-wise Gate**



*Original relation messages + Gated template messages → Next-layer representation*

# Method Overview (Recap)

## ■ Model Operates on Time-Consistent Computational Graph



# Definition: Desired Graph $G_{\phi_g^*}$

$\phi_g$  denote structural parameters

$$G_{\phi_g^*} \text{ whose structure parameter } \phi_g^* \in \arg \min_{\phi_g} \left\{ \underbrace{\mathcal{R}^*(\phi_g)}_{\text{Regularized Risk}} + \underbrace{\Omega(\phi_g)}_{\text{Structural complexity}} \right\}$$

## Regularized Risk



Task Prediction Loss + VIB Compression Penalty

## Structural Complexity

# Active Columns + # Active Tables + # Active Templates

*Learn a graph structure that balances **predictive performance** and **structural simplicity***

# Method: Learning the Desired Graph

## ■ Loss Function

$$\mathcal{L} = \underbrace{\mathcal{L}_{\text{task}}(\hat{Y}, Y)}_{\text{Task Prediction Loss}} + \beta_{\text{vib}} \sum_{T_i \in \mathcal{T}} \frac{1}{|T_i|} \sum_{v \in T_i} \text{KL}\left(q(z_v | \tilde{h}_v) \parallel \mathcal{N}(\mathbf{0}, \mathbf{I})\right)$$

Regularized Risk

VIB Compression Penalty

$$+ \underbrace{\lambda}_{\text{Overall Filtering Strength}} \left( \sum_{T_i \in \mathcal{T}} \mathbb{E}[\|g_{T_i}\|_0] + \lambda_s \sum_{T_i \in \mathcal{T}} \mathbb{E}[\|s_{T_i}\|_0] \right) + \underbrace{\lambda_k}_{\text{Injection Sparsity Strength}} \sum_{k=1}^{K_{\text{temp}}} \mathbb{E}[\|g_k\|_0]$$

Structural Complexity

Active Columns

Active Tables

Active Templates

Overall Filtering Strength

Injection Sparsity Strength

# Empirical Analysis

## ■ Filtering

| Methods    | Classification<br>(ROC-AUC (%), ↑) |                  |                  |                  | Regression<br>(MAE, ↓) |                    |                    | Recommendation<br>(MAP, ↑) |                      |
|------------|------------------------------------|------------------|------------------|------------------|------------------------|--------------------|--------------------|----------------------------|----------------------|
|            | study-outcome(↑)                   | driver-top3(↑)   | user-repeat(↑)   | user-clicks(↑)   | study-adverse(↓)       | driver-position(↓) | user-attendance(↓) | condition-sponsor-run(↑)   | post-post-related(↑) |
| Base       | 69.44 $\pm$ 0.50                   | 81.48 $\pm$ 1.06 | 78.70 $\pm$ 0.05 | 65.44 $\pm$ 0.70 | 45.50 $\pm$ 0.14       | 3.88 $\pm$ 0.02    | 0.24 $\pm$ 0.01    | 2.83 $\pm$ 0.01            | 1.24 $\pm$ 0.08      |
| RandFilter | 63.21 $\pm$ 1.14                   | 75.36 $\pm$ 1.75 | 73.27 $\pm$ 1.66 | 61.54 $\pm$ 1.82 | 48.23 $\pm$ 1.17       | 4.85 $\pm$ 0.87    | 0.47 $\pm$ 0.05    | 2.41 $\pm$ 0.01            | 0.41 $\pm$ 0.08      |
| VarFilter  | 69.91 $\pm$ 0.43                   | 82.07 $\pm$ 0.81 | 78.16 $\pm$ 0.12 | 66.04 $\pm$ 0.57 | 45.18 $\pm$ 0.11       | 3.51 $\pm$ 0.03    | 0.24 $\pm$ 0.01    | 2.91 $\pm$ 0.01            | 1.63 $\pm$ 0.05      |
| ColFilter  | 69.87 $\pm$ 0.01                   | 83.64 $\pm$ 0.04 | 79.95 $\pm$ 0.82 | 66.31 $\pm$ 0.18 | 43.83 $\pm$ 0.38       | 3.83 $\pm$ 0.03    | 0.23 $\pm$ 0.01    | 3.12 $\pm$ 0.01            | 1.74 $\pm$ 0.01      |
| FullFilter | 70.85 $\pm$ 0.46                   | 84.66 $\pm$ 0.61 | 80.47 $\pm$ 0.89 | 67.17 $\pm$ 0.06 | 43.60 $\pm$ 0.05       | 3.79 $\pm$ 0.16    | 0.23 $\pm$ 0.01    | 3.63 $\pm$ 0.01            | 1.77 $\pm$ 0.01      |

## ■ Injection

| Methods     | Classification   |                  |                  |                  | Regression       |                  | Recommendation    |                   | Improvement Range |
|-------------|------------------|------------------|------------------|------------------|------------------|------------------|-------------------|-------------------|-------------------|
|             | st-out           | d-t3             | u-rep            | u-clk            | st-adv           | d-pos            | c-sp-r            | p-p-r             |                   |
| Base        | 69.44<br>(0%)    | 81.48<br>(0%)    | 78.70<br>(0%)    | 65.44<br>(0%)    | 45.50<br>(0%)    | 3.88<br>(0%)     | 2.83<br>(0%)      | 1.42<br>(0%)      |                   |
| KNN         | 70.93<br>(+2.1%) | 86.67<br>(+6.4%) | 81.60<br>(+3.7%) | 66.61<br>(+1.8%) | 44.10<br>(-3.1%) | 3.65<br>(-5.9%)  | 4.85<br>(+71.4%)  | 3.47<br>(+144.4%) | 10-20%            |
| 2Hop        | 70.33<br>(+1.3%) | 82.19<br>(+0.9%) | 78.79<br>(+0.1%) | 65.79<br>(+0.5%) | 44.15<br>(-3.0%) | 3.98<br>(+2.6%)  | 2.91<br>(+2.8%)   | 1.53<br>(+7.7%)   | 5-10%             |
| BehaviorSim | 70.06<br>(+0.9%) | 82.95<br>(+1.8%) | 79.14<br>(+0.6%) | 66.30<br>(+1.3%) | 44.22<br>(-2.8%) | 3.69<br>(-4.9%)  | 6.24<br>(+120.5%) | 8.51<br>(+499.3%) | 1-5%              |
| TempCont    | 70.61<br>(+1.7%) | 84.42<br>(+3.6%) | 80.04<br>(+1.7%) | 66.83<br>(+2.1%) | 41.28<br>(-9.3%) | 2.14<br>(-44.8%) | 4.76<br>(+68.2%)  | 4.35<br>(+206.3%) | 0-1%              |

# Experiments: Predictive Performance



## Classification (ROC-AUC (%), ↑)

| Method     | study-outcome    | driver-dnf       | driver-top3      | user-repeat      | user-ignore      | user-clicks      | user-visits      |
|------------|------------------|------------------|------------------|------------------|------------------|------------------|------------------|
| Base       | 69.44 $\pm$ 0.53 | 74.53 $\pm$ 0.27 | 81.48 $\pm$ 1.06 | 78.75 $\pm$ 0.62 | 75.39 $\pm$ 0.54 | 65.44 $\pm$ 0.71 | 63.82 $\pm$ 0.67 |
| LightGBM   | 65.27 $\pm$ 0.81 | 69.13 $\pm$ 1.14 | 76.09 $\pm$ 0.93 | 74.51 $\pm$ 0.72 | 74.58 $\pm$ 0.71 | 61.27 $\pm$ 0.93 | 63.03 $\pm$ 0.85 |
| HAN        | 69.05 $\pm$ 0.67 | 74.05 $\pm$ 1.08 | 82.09 $\pm$ 0.93 | 79.24 $\pm$ 0.75 | 79.27 $\pm$ 0.76 | 66.15 $\pm$ 0.63 | 66.57 $\pm$ 0.74 |
| GTN        | 68.15 $\pm$ 0.47 | 72.36 $\pm$ 0.68 | 82.50 $\pm$ 0.63 | 79.85 $\pm$ 0.61 | 78.62 $\pm$ 0.77 | 64.85 $\pm$ 0.69 | 66.82 $\pm$ 0.34 |
| HGSL       | 69.51 $\pm$ 0.47 | 71.56 $\pm$ 0.48 | 83.51 $\pm$ 0.75 | 80.64 $\pm$ 0.81 | 77.23 $\pm$ 1.07 | 63.42 $\pm$ 0.86 | 67.08 $\pm$ 0.71 |
| REL-GNN    | 70.86 $\pm$ 0.54 | 75.41 $\pm$ 0.29 | 82.71 $\pm$ 0.83 | 79.08 $\pm$ 0.81 | 80.31 $\pm$ 0.86 | 65.91 $\pm$ 0.82 | 68.14 $\pm$ 0.76 |
| RelGT      | 69.74 $\pm$ 0.32 | 76.33 $\pm$ 0.58 | 83.14 $\pm$ 0.75 | 79.46 $\pm$ 0.32 | 81.24 $\pm$ 0.77 | 66.87 $\pm$ 0.60 | 68.94 $\pm$ 0.65 |
| LLM-Struct | 70.83 $\pm$ 0.48 | 76.80 $\pm$ 0.39 | 83.57 $\pm$ 0.37 | 80.13 $\pm$ 0.77 | 79.90 $\pm$ 0.73 | 68.75 $\pm$ 0.37 | 69.25 $\pm$ 0.39 |
| Ours       | 72.35 $\pm$ 0.37 | 78.21 $\pm$ 0.15 | 86.82 $\pm$ 0.61 | 82.36 $\pm$ 0.55 | 81.25 $\pm$ 0.64 | 67.53 $\pm$ 0.15 | 70.15 $\pm$ 0.56 |

## Regression (MAE, ↓)

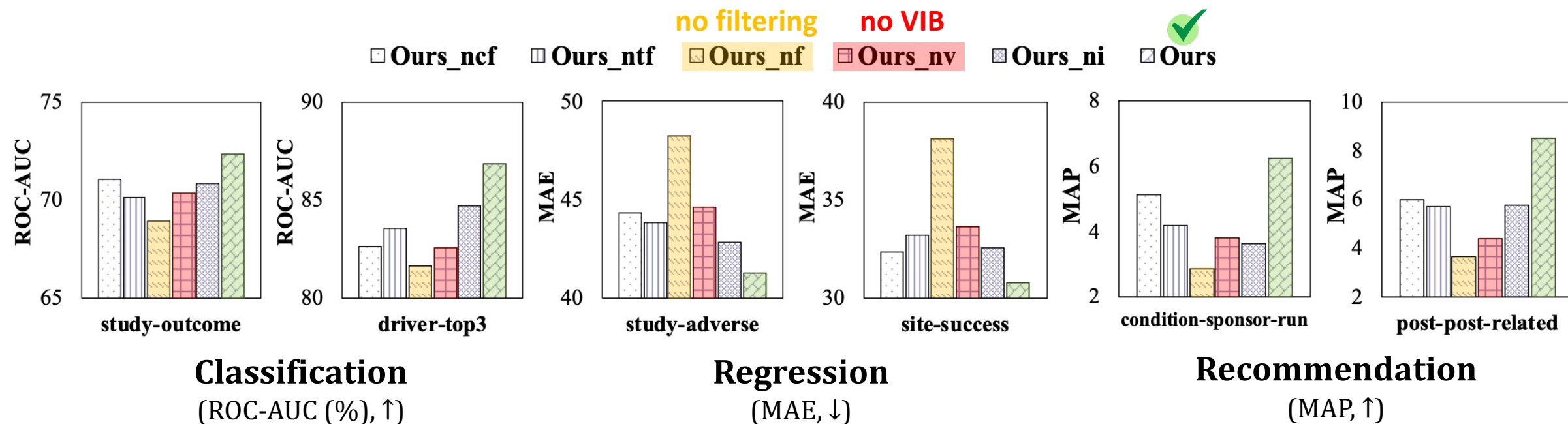
| Method     | study-adverse | site-success | driver-position | user-attendance | ad-ctr |
|------------|---------------|--------------|-----------------|-----------------|--------|
| Base       | 45.50         | 38.86        | 3.88            | 0.24            | 0.231  |
| LightGBM   | 48.31         | 0.38         | 4.95            | 0.51            | 0.242  |
| HAN        | 45.16         | 0.35         | 3.76            | 0.24            | 0.228  |
| GTN        | 44.82         | 0.36         | 3.70            | 0.23            | 0.225  |
| HGSL       | 44.69         | 0.36         | 3.85            | 0.25            | 0.246  |
| REL-GNN    | 44.93         | 0.34         | 3.78            | 0.23            | 0.227  |
| RelGT      | 44.62         | 0.32         | 3.72            | 0.23            | 0.225  |
| LLM-Struct | 44.73         | 0.34         | 3.69            | 0.23            | 0.228  |
| Ours       | 41.28         | 0.31         | 2.13            | 0.23            | 0.221  |

| Method     | user-ad-visit | user-post-comment | post-post-related | condition-sponsor-run | site-sponsor-run |
|------------|---------------|-------------------|-------------------|-----------------------|------------------|
| Base       | 2.31          | 8.77              | 1.24              | 2.83                  | 10.21            |
| LightGBM   | 1.14          | 6.26              | 0.41              | 2.15                  | 8.35             |
| HAN        | 2.33          | 11.06             | 1.31              | 2.91                  | 15.27            |
| ID-GNN     | 3.24          | 13.41             | 10.61             | 11.65                 | 18.25            |
| REL-GNN    | 2.87          | 13.12             | 2.43              | 3.47                  | 17.31            |
| LLM-Struct | 3.28          | 13.64             | 4.72              | 4.62                  | 18.71            |
| Ours       | 3.84          | 14.82             | 8.53              | 6.24                  | 19.15            |

## Recommendation (MAP, ↑)

# Ablation Study

■ VIB provides a modest but consistent benefit



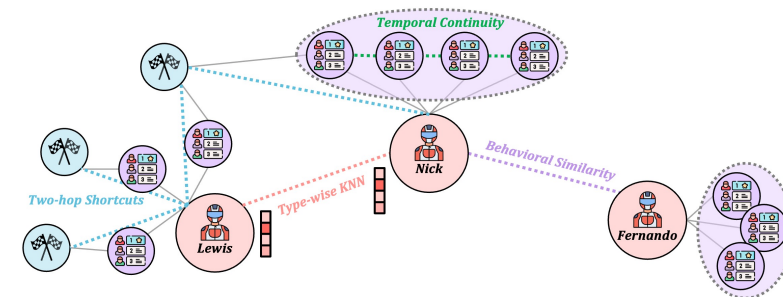
## ■ Potential Computational Overhead of Structural Injection

- Injection is performed on **sampled time-consistent subgraphs**, not the full relational entity graph

$$G_s = (V_s, E_s) \quad n = |V_s|, \quad m = |E_s|$$

| Type-wise KNN                  | Behavioral Similarity                         | Two-hop Shortcuts                                   | Temporal Continuity                 |
|--------------------------------|-----------------------------------------------|-----------------------------------------------------|-------------------------------------|
| $O\left(\sum_T n_T^2 d\right)$ | $O(m_r d + n_A^2 d)$                          | $O\left(\sum_{b \in B} d_{in}(b) d_{out}(b)\right)$ | $O\left(\sum_e r_e \log r_e\right)$ |
| Pairwise Similarity            | Neighbor Aggregation<br>+ Pairwise Similarity | High-degree Middle Nodes                            | Sorting                             |

- For reference: Standard GNN message passing  $O(md)$



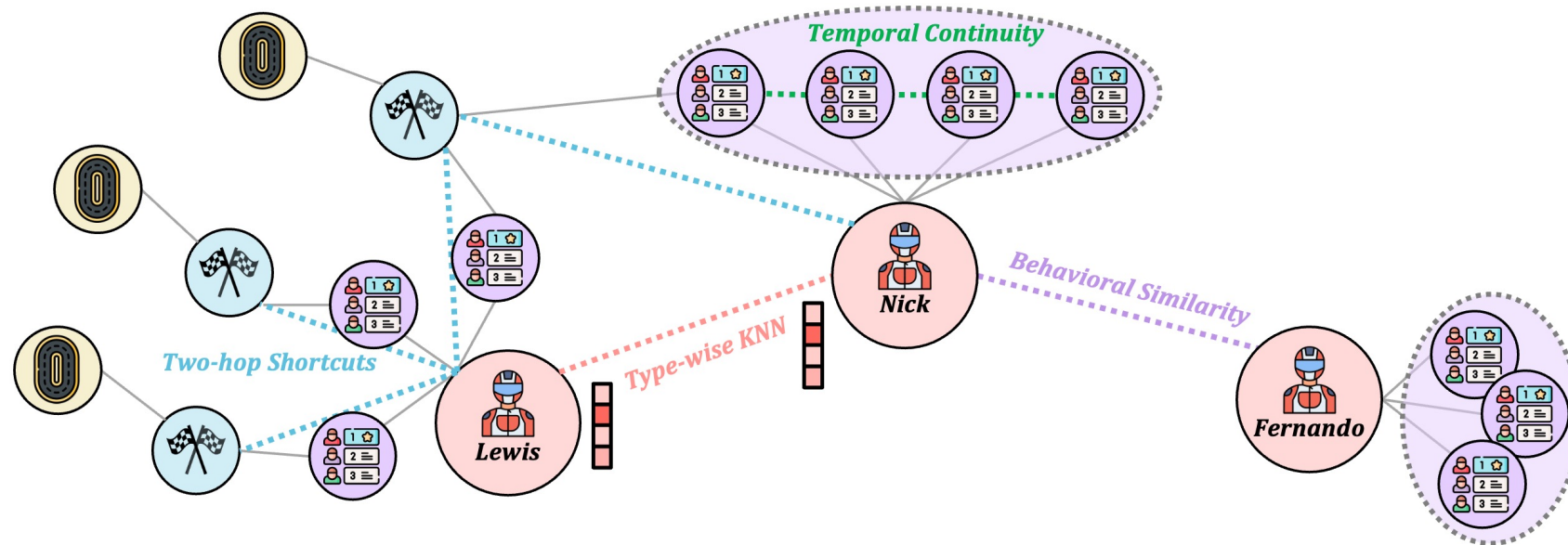
## ■ Expressiveness of Structural Injection

### □ Template Space Limitation

- Limited to predefined structural patterns; unseen relational structures cannot be discovered

### □ Sampling before Injection

- Candidate nodes are determined by schema-based sampling before injection
- Potential mismatch between schema hop distance and relational relevance



# Conclusion

## Traditional Approach to RDB

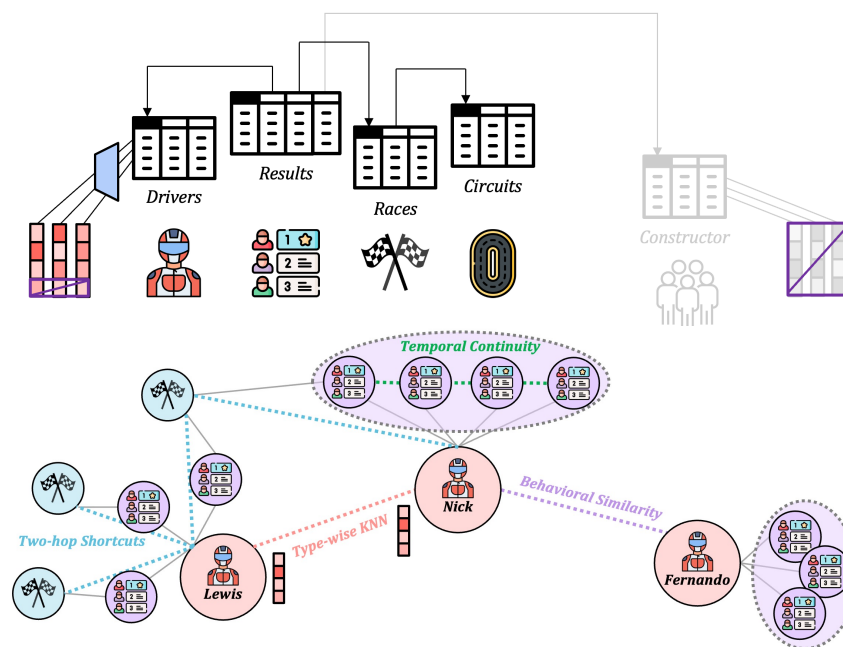
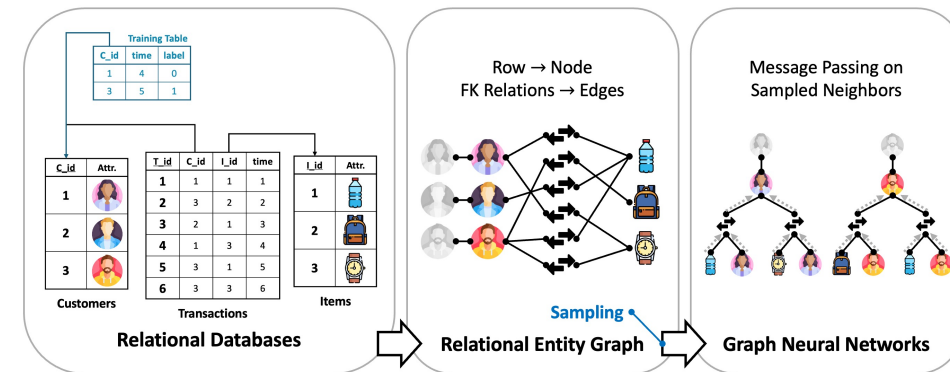
- Transform relational data into a single-table
- Limitation: Manual feature engineering, Loss of relational structure

## Relational Deep Learning

- Key Idea: Represent RDB as a REG and apply GNNs
- Our Focus: Graph Representation

## What Makes a Desired Graph for Relational Deep Learning?

- Motivation: Information overload and Semantic fragmentation
- Key Idea: Information filtering and Structural injection
- Limitation: Potential Computational Overhead & Limited Expressiveness



# Q&A